Machine Learning Analysis of Spectral Data Analysis

Imama Jawad
School of Electronics, Electrical Engineering and Computer Science
Queen's University Belfast
40462364

November 11, 2024

Contents

1 Introduction

1.1 Dataset Description

This dataset was collected as part of a research project funded to explore virus detection through spectroscopy. The data is organized into several columns, each capturing critical information and measurements from multiple virus testing sessions. Below is an overview of the columns:

- **SID (Session ID)**: A distinct identifier for each data collection session. For example, the identifier "S1" appears in multiple rows, signifying that these rows are from the same session.
- **Type (Virus Type)**: Defines the specific virus being tested, with the categories "X" and "Y".
- **Matrix (Medium)**: Describes the medium in which the virus sample is prepared during testing. The value *dmem* refers to Dulbecco's Modified Eagle Medium, which supports the growth of mammalian cells and is essential for virus propagation. The value *pbs* refers to Phosphate-Buffered Saline, a commonly used buffer solution to stabilize pH and for tasks like washing cells or diluting samples.
- **Load (Dilution Level)**: Indicates the extent of serial dilution applied to the virus sample. A higher dilution level means a lower concentration of the virus.
- **I001 to I512 (Light Intensity at Different Wavelengths)**: These columns contain numerical readings of light intensity measured at various wavelengths.

[5]:
```
data=pd.read_csv("Ml Tutorials/2022QUB (1).csv")
```

[6]:
```
data.head()
```

[6]:

	SID	Type	Matrix	Load	I001	I002	I003	I004	I005	I006	...	I503	I504	I505	I506	I507	I508	I509	I510	I511	I512
0	S1	X	dmem	1	97.78	97.73	97.68	97.70	97.73	97.73	...	83.38	84.49	84.94	85.47	86.16	86.79	87.12	87.20	87.58	88.20
1	S1	X	dmem	1	97.75	97.70	97.70	97.71	97.71	97.72	...	83.65	84.77	85.34	85.27	85.69	85.52	85.27	85.28	85.41	85.65
2	S1	X	dmem	1	97.76	97.72	97.70	97.74	97.71	97.71	...	84.46	85.38	86.03	86.38	86.77	87.10	87.14	86.89	87.14	87.79
3	S1	X	dmem	1	97.72	97.71	97.71	97.73	97.70	97.71	...	83.96	84.89	85.43	85.30	85.80	86.18	86.39	86.63	87.21	87.76
4	S1	X	dmem	1	97.74	97.66	97.65	97.68	97.67	97.65	...	83.65	84.02	84.33	84.53	85.14	85.40	85.41	85.42	85.88	85.87

5 rows × 516 columns

[7]:
```
data.shape
```

[7]:
```
(3801, 516)
```

Figure 1: Data Loading, Head and Shape

In the above code, the dataset is loaded, and a few rows are looked at using the head function. The shape function tells us that there are 3801 total rows (samples) and 516 total columns: (SID, Type, Matrix, Load, and the 512 Light intensity at different wavelength columns)

2 Experimental Design

This section delves into the three major tasks done on Viral Dataset in this report. This includes Problem Descriptions, Data Preprocessing, Feature Engineering, and the Modelling Approach.

2.1 Problem Description

Did Three major tasks which include:

- **Regression**: Developed a regression model to estimate the Load or dilution applied given the rest of the spectroscopy data.
- **Classification**: Performed a classification task to determine the virus type (x or Y) being tested from the spectroscopy data.
- **Clustering**: Grouped the Data into clusters to predict Load based on the mean of each cluster.

Each task is significant for analyzing and understanding viral behavior.

2.2 Data Preprocessing and Feature Engineering

To ensure model reliability and accuracy, the dataset underwent extensive preprocessing steps, including:

2.2.1 Check for Missing Values

: A quick check was performed to identify any Null, NA, or empty values in both categorical and numerical features. The `isnull()` function was used to check for nulls, and a glance over the unique values in categorical columns was done to identify any blanks. No such cases were found in the data. Since no imputation was performed at this stage, this step will not cause any data leakage during the subsequent train-test split.

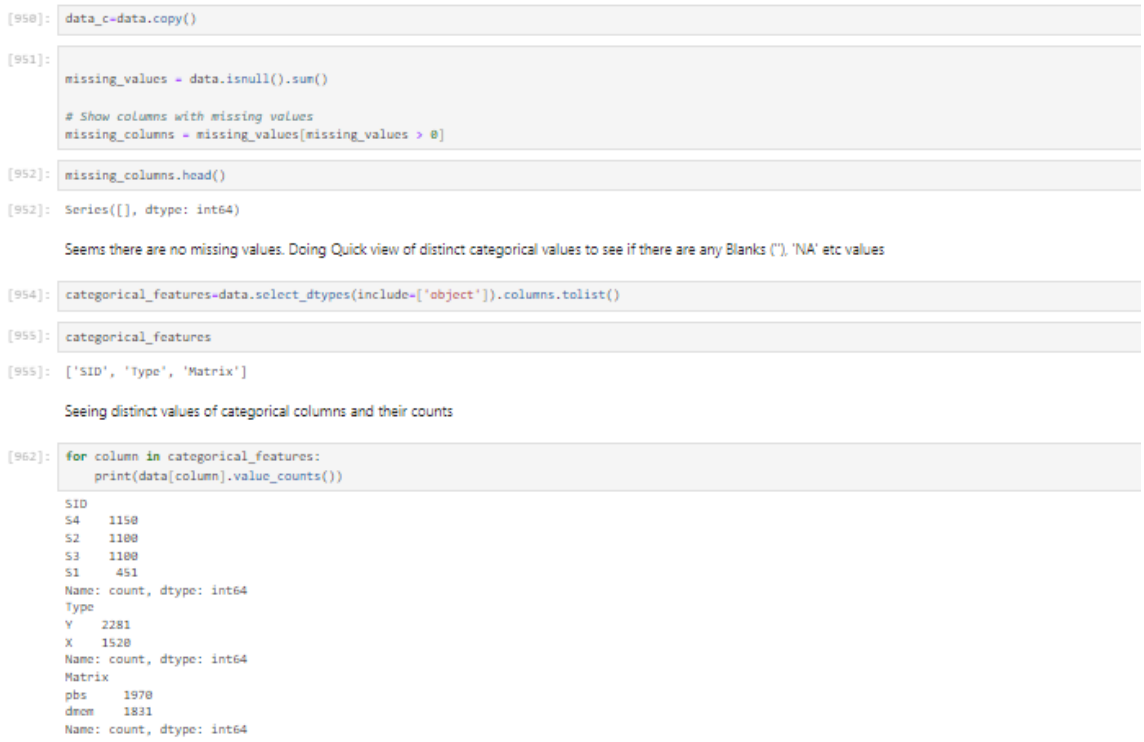


Figure 2: Check for Empty and NULL Data

2.2.2 Encoding Categorical Columns

: Categorical features were encoded using `One-Hot Encoding`, as shown in Figure **??**. Given the non-ordinal distinct values for each feature, one-hot encoding was the best approach [2]. For features with two values, binary mapping classified them as 0 or 1. The SID column, with four values, was transformed into three columns, while Type (X, Y) and Matrix (Pbs, dmem) were binary-mapped to TypeX and MatrixPbs. Since One-Hot Encoding creates independent binary columns without relying on the target or other data points, it avoids data leakage even when applied before data splitting.[1]

One-Hot Encoding

```
#no NA's as such or empty vaalues
#Not a lot of categories, most of these features seem not ordinal and binary, so can convert yes no features to 1,0 as 1'st step-
```

```
## removing whitestrip or spaces in front of letters
for column in categorical_features:
    data[column] = data[column].str.strip()
```

Not a lot of categories, most of these features seem non ordinal and binary, so can convert features with two disctnct values as o or 1;Other non ordinal features with multiple categories with one hot encoding

```
binary_map = {'X': 1, 'Y': 0}
data['Type']=data['Type'].map(binary_map)
```

```
#renmaing column to make sense
data=data.rename(columns={"Type": "TypeX"})
```

```
binary_map = {'pbs': 1, 'dmem': 0}
data['Matrix']=data['Matrix'].map(binary_map)
data=data.rename(columns={"Matrix": "MatrixPbs"})
```

```
data = pd.get_dummies(data, columns=['SID'], drop_first=True)
```

```
data.head()
```

	TypeX	MatrixPbs	Load	I001	I002	I003	I004	I005	I006	I007	...	I506	I507	I508	I509	I510	I511	I512	SID_S2	SID_S3	SID_S4
0	1	0	1	97.78	97.73	97.68	97.70	97.73	97.73	97.72	...	85.47	86.16	86.79	87.12	87.20	87.58	88.20	False	False	False
1	1	0	1	97.75	97.70	97.70	97.71	97.71	97.72	97.74	...	85.27	85.69	85.52	85.27	85.28	85.41	85.65	False	False	False
2	1	0	1	97.76	97.72	97.70	97.74	97.71	97.71	97.74	...	86.38	86.77	87.10	87.14	86.89	87.14	87.79	False	False	False
3	1	0	1	97.72	97.71	97.71	97.73	97.70	97.71	97.70	...	85.30	85.80	86.18	86.39	86.63	87.21	87.76	False	False	False
4	1	0	1	97.74	97.66	97.65	97.68	97.67	97.65	97.69	...	84.53	85.14	85.40	85.41	85.42	85.88	85.87	False	False	False

5 rows × 518 columns

Figure 3: Categorical Feature Encoding

2.2.3 Defining Features & Target

:

- **For Regression Task**:
  - The target variable is set to the 'Load' column.
  - All other columns are added to the feature set X.
- **For Classification Task**:
  - The target variable is set to the 'TypeX' column, representing the virus type.
  - All other columns are added to the feature set X.
- **For Clustering Task**:
  - There is no target variable since it is an unsupervised learning task.
  - The 'Load' column is removed from the feature set X.

2.2.4 Splitting Data into Train and Test Data

:

- **For Regression Task**:
  - Data is split into 70 Percent Train data, 15 Percent Validation data, 15 Percent Test Data. This Validation Dataset is used further in hyper parameter tuning. Furthermore, data was stratified on Load as we have 8 different values of Load and each class has imbalanced distribution so wanted to represent equal distribution in train, test and validation data

```
[985]: #Doing a split of 70% train, 15% test, 15% validation

[989]: # stratify on Load

[990]: # Split the data into train (70%) and test (30%), stratified on 'Viral_Load'
       X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

       # Further split the test set into validation (50%) and test (50%), stratified on 'Viral_Load'
       X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, random_state=42, stratify=y_test)
```

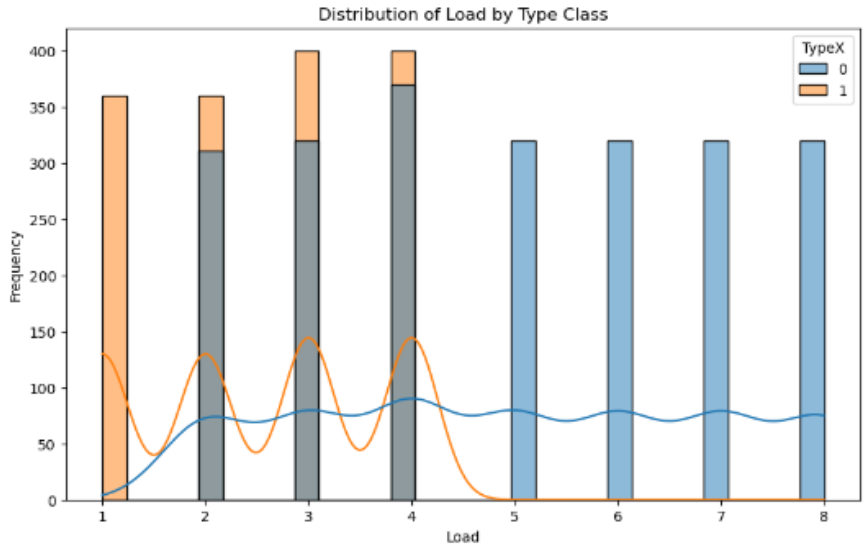
Figure 4: Splitting on Regression- Stratifying on Load [4]

- **For Classification Task**:

– Data is split into 70 Percent Train data, 30 Percent Test Data. Cross Validation is used later within Training Data in this Model instead of separating validation dataset initially. Furthermore I stratified on Load and Feature columns combined so train and test data has equal distribution.

```
[569]: data_all_new=pd.concat([X_all, y_all], axis=1)


plt.figure(figsize=(10, 6))
sns.histplot(data=data_all_new, x='Load', hue='TypeX', bins=30, kde=True)
plt.title('Distribution of Load by Type Class')
plt.xlabel('Load')
plt.ylabel('Frequency')
plt.show()
```




```
[570]: #Want to stratify on Load and TypeX column when splitting Data
stratify_labels = data_all_new['Load'].astype(str) + "_" + data_all_new['TypeX'].astype(str)
```

```
[571]: X_train_new, X_test_new, y_train_new, y_test_new = train_test_split(X_all, y_all, test_size=0.3, stratify=stratify_labels, random_state=42)

print("New Training Set:", X_train_new.shape, y_train_new.shape)
print("New Test Set:", X_test_new.shape, y_test_new.shape)
```

Figure 5: Classification Split and stratify [4]

- **For Clustering Task**:
  - There is no splitting into train, test here for clustering. Just using of mean Load cluster to predict Load and calculating mse on whole data

2.2.5 Scaling and Normalization

: The Wavelength columns I001: I512 were Scaled using MinMax Scaler.First normality of features was tested using Shapiro test, no features were normally distributed, so MinMax scaling made sense in this case[2]. As the rest of the other features only had 0, 1 values there was no use for scaling for the others them.

```
normality_results = {}
for column in X_train.columns:
    stat, p_value = shapiro(X_train[column])
    normality_results[column] = p_value
    if p_value > 0.05:
        print(f"{column} is likely normally distributed (p-value: {p_value})")
```

[865]:
```
columnsLightWavelengths = [col for col in data.columns if col.startswith('I')]
```

as not normally distributed going with MinMax Scaler

[876]:
```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train.loc[:, columnsLightWavelengths] = scaler.fit_transform(X_train[columnsLightWavelengths])
X_val.loc[:, columnsLightWavelengths] = scaler.transform(X_val[columnsLightWavelengths])
X_test.loc[:, columnsLightWavelengths]= scaler.transform(X_test[columnsLightWavelengths])
```

[878]:
```
X_train.describe()
```

[878]:

	TypeX	MatrixPbs	I001	I002	I003	I004	I005	I006	I007	I008	...	I503	I504
count	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	2660.000000	...	2660.000000	2660.000000
mean	0.401128	0.516165	0.692269	0.695870	0.699010	0.699223	0.699149	0.701073	0.703026	0.704835	...	0.755993	0.749333
std	0.490219	0.499833	0.131330	0.131894	0.131991	0.132195	0.131993	0.132990	0.134173	0.134089	...	0.116362	0.117416
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000
25%	0.000000	0.000000	0.602733	0.607197	0.610041	0.608789	0.608569	0.608163	0.609665	0.612978	...	0.695016	0.685230
50%	0.000000	1.000000	0.685729	0.689052	0.692698	0.693516	0.692619	0.695153	0.696392	0.698552	...	0.776815	0.771902
75%	1.000000	1.000000	0.761134	0.764318	0.768763	0.769504	0.769590	0.772449	0.776289	0.778309	...	0.834778	0.830796
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	...	1.000000	1.000000

8 rows × 514 columns

Figure 6: Scaling

2.2.6 Dimensionality Reduction

Principal Component Analysis (PCA) was applied to reduce the feature dimensionality for the light Wavelength columns while retaining around 99% of the variance. This was achieved through using just 2 components. The dimensionality for the wavelength columns reduced from 512 to 2. The original wavelength columns were removed from all sets. This step was crucial for computational efficiency and model performance.

```
# Select only the columns corresponding to light wavelengths from X_train
X_train_light_wavelengths = X_train[columnsLightWavelengths]
X_test_light_wavelengths = X_test[columnsLightWavelengths]
X_val_light_wavelengths = X_val[columnsLightWavelengths]
# Apply PCA only to the light wavelength columns
n_components_1 = 2 # Number of components for PCA
pca_column_names = [f"PCA_Component_{i+1}" for i in range(n_components_1)]
# Initialize PCA
pca = PCA(n_components=n_components_1)
# Fit PCA on X_train_light_wavelengths and transform it
X_train_light_wavelengths_reduced = pca.fit_transform(X_train_light_wavelengths)
# Transform X_test and X_val using the same PCA transformation
X_test_light_wavelengths_reduced = pca.transform(X_test_light_wavelengths)
X_val_light_wavelengths_reduced = pca.transform(X_val_light_wavelengths)
```

Figure 7: Initializing PCA

```
•[65]: # Now, combine the reduced light wavelength features with the other columns from X_train, X_test, and X_val
X_train_combined = X_train.drop(columns=columnsLightWavelengths)
X_train_combined = pd.concat([X_train_combined, pd.DataFrame(X_train_light_wavelengths_reduced,index=X_train_combined.index,columns=pca_column_names)],
X_test_combined = X_test.drop(columns=columnsLightWavelengths)
X_test_combined = pd.concat([X_test_combined, pd.DataFrame(X_test_light_wavelengths_reduced,index=X_test_combined.index,columns=pca_column_names)], axis
X_val_combined = X_val.drop(columns=columnsLightWavelengths)
X_val_combined = pd.concat([X_val_combined, pd.DataFrame(X_val_light_wavelengths_reduced,index=X_val_combined.index,columns=pca_column_names)], axis=1)
# Output the shapes of the reduced datasets
print("Reduced features shape (X_train):", X_train_combined.shape)
print("Reduced features shape (X_test):", X_test_combined.shape)
print("Reduced features shape (X_val):", X_val_combined.shape)
# Explained variance calculation
explained_variance = pca.explained_variance_ratio_
cumulative_explained_variance = np.cumsum(explained_variance)
# Plot the explained variance for the light wavelength columns
plt.figure(figsize=(3, 3))
plt.bar(range(1, n_components_1+1), explained_variance[:n_components_1], alpha=0.5, align='center', label='Explained variance')
plt.step(range(1, n_components_1+1), cumulative_explained_variance[:n_components_1], where='mid', label='Cumulative explained variance')
plt.xlabel('Principal Components')
plt.ylabel('Explained Variance Ratio')
plt.title('Explained Variance (Light Wavelengths)')
plt.legend(loc='best')
# Display the plot
plt.tight_layout()
plt.show()
```

```
Reduced features shape (X_train): (2660, 7)
Reduced features shape (X_test): (571, 7)
Reduced features shape (X_val): (570, 7)
```

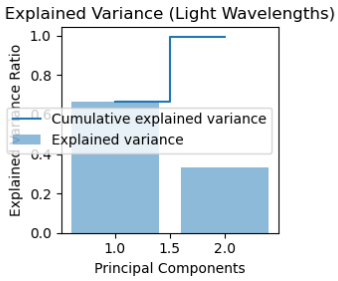


Figure 8: Viewing Cummalative Explained Variance

First component seems to be mostly mapped from 1001 to I300, The second component is mapped mostly from 1300 onwards

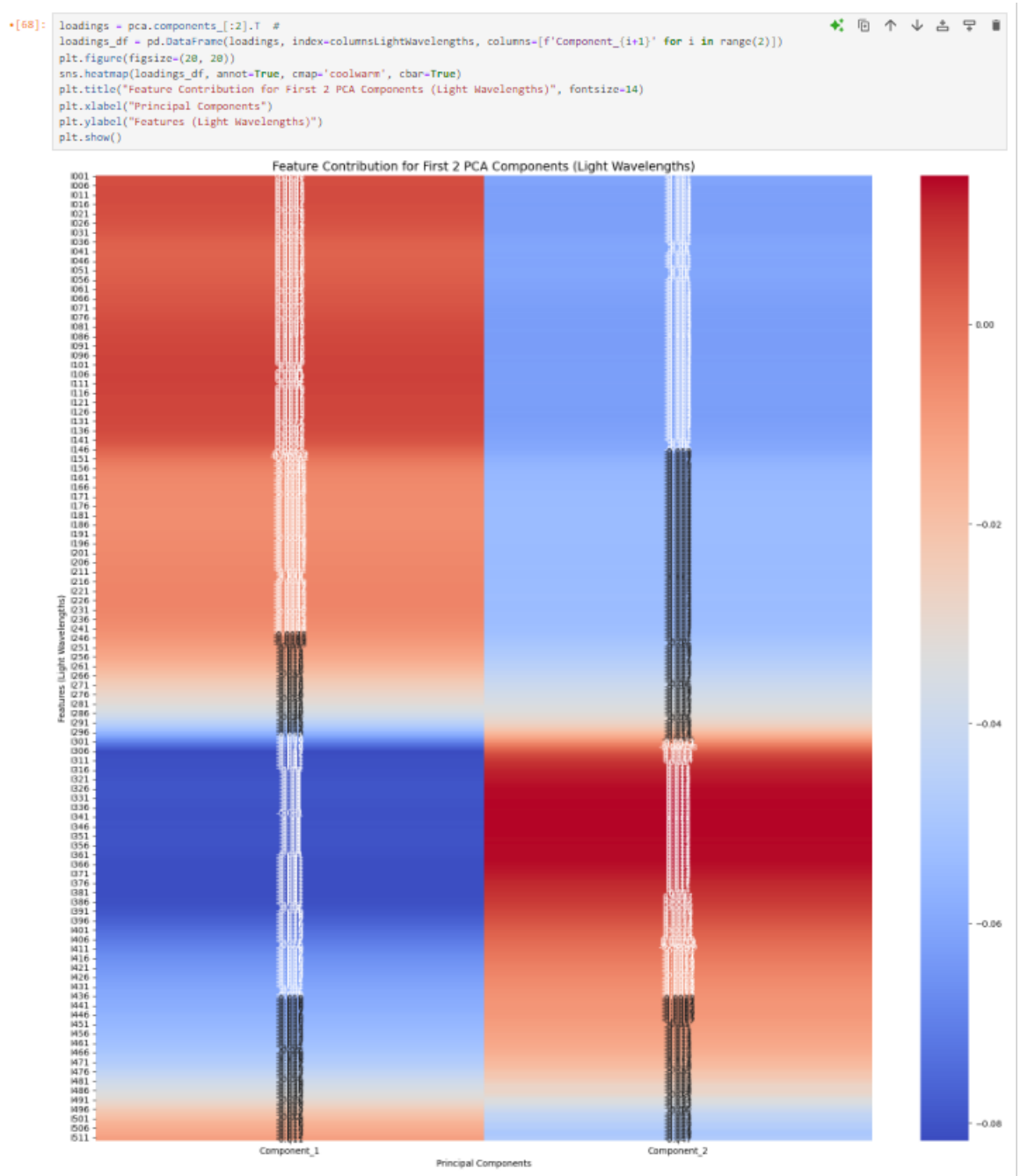


Figure 9: Seeing Feature contribution of wavelength columns in PCA components for wavelengths

2.2.7 Outlier Analysis

: Outliers were detected using z-scores. Upper, lower bounds were calculated on Just Train data. For handling outliers I basically capped outliers to upper or lower bounds. We then performed the same treatment for Test, Validation datasets accordingly based on the Upper lower bounds calculated on training data. This ensured no Data leakage [1]

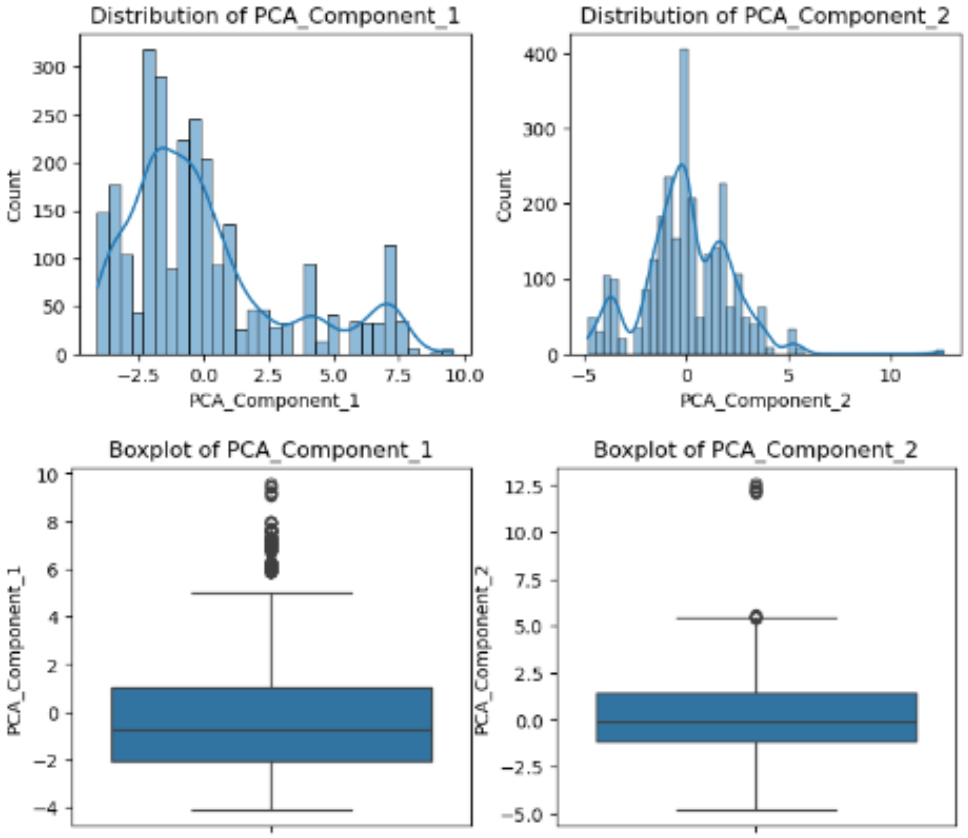


Figure 10: Boxplot of PCA Wavelength features before catering for outliers

```
[72]:
for col in PCA_Features:
    # Calculate the first (Q1) and third (Q3) quartiles, and the IQR (Interquartile Range)
    Q1 = X_train_combined[col].quantile(0.25)
    Q3 = X_train_combined[col].quantile(0.75)
    IQR = Q3 - Q1

    # Calculate the lower and upper bounds for detecting outliers
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap outliers in the training set: any value below lower_bound is set to lower_bound,
    # and any value above upper_bound is set to upper_bound
    X_train_combined[col] = X_train_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_bound else x)
    )

    # Apply the same capping to the validation and test sets using the training data's bounds
    X_val_combined[col] = X_val_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_bound else x)
    )

    X_test_combined[col] = X_test_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_bound else x)
    )
```

```
[73]:
# Boxplots aftter removing outliers
plt.figure(figsize=(15, 12))
for i, column in enumerate(PCA_Features, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(y=X_train_combined[column])
    plt.title(f'Boxplot of {column}')
plt.tight_layout()
plt.show()
```

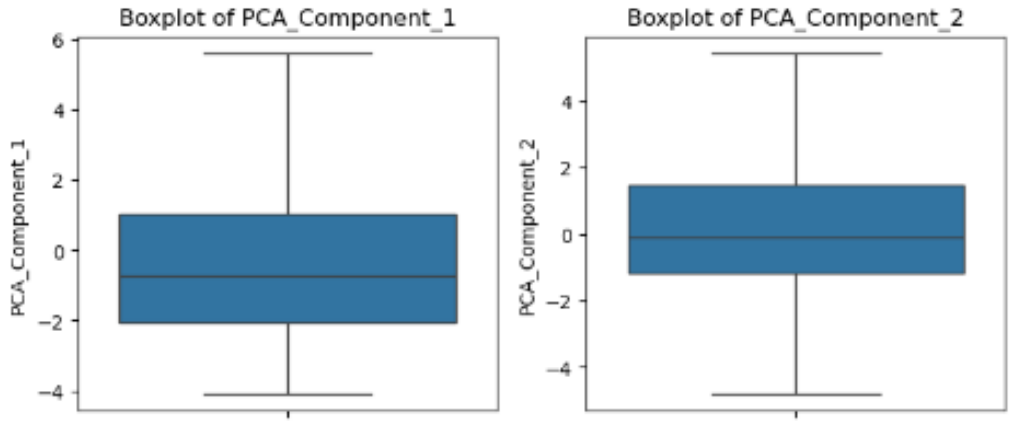


Figure 11: BoxPlot of PCA Wavelength features after catering for outliers

2.2.8 Further Feature Engineering and Correlation heatmaps

:

For Regression Task:

- No further features were created after PCA of wavelength features as correlation heatmap was showing some features with high correlation with target variable and sufficiently low inter-feature correlation. Type column is 58 Percent correlated with Target column

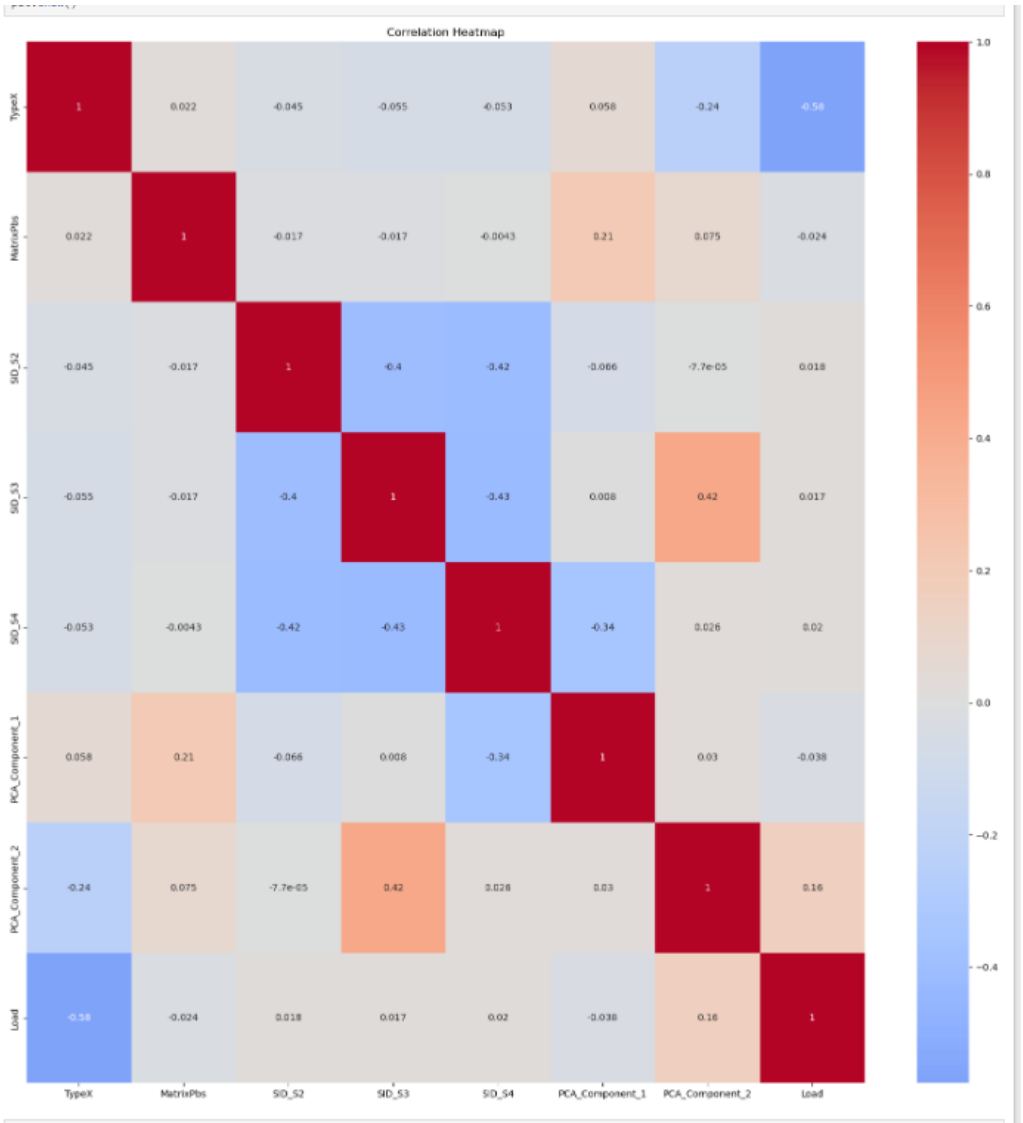


Figure 12: Correlation Heatmap Regression task

For Classification Task:

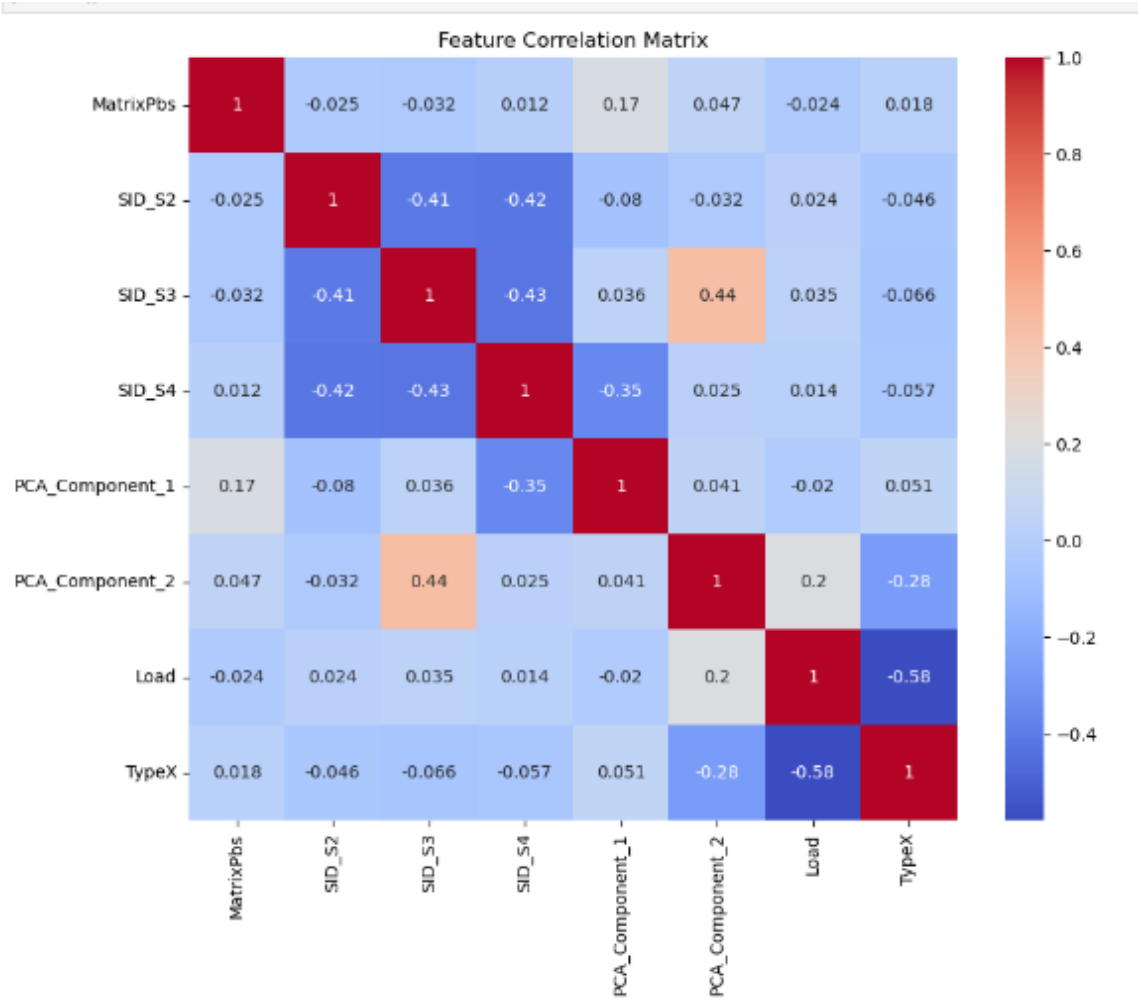


Figure 13: Correlation before further Feature Engineering

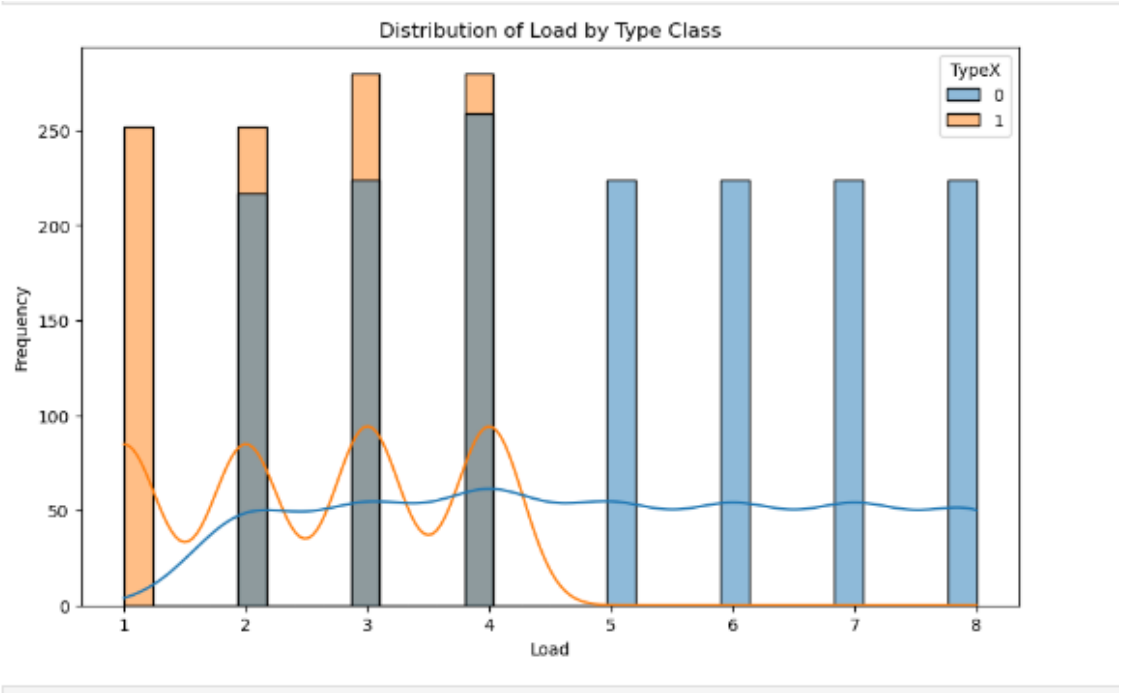


Figure 14: Distribution of Type against Load

- As Loads 5 onwards have just Type Y, so makes sense to bin this feature

```
[583]:
# we can see in training data Loads 5-8 have just Type Y, Load 1 just has Type X , Loads 2-4 are the ones really which have some variation in X
bins = [-np.inf, 1, 2,3,4, np.inf]  # Example bin edges, adjust these based on plot
labels = [1, 2,3, 4, 5]

# Create the binned_load feature
X_train_new['LoadCapped5'] = pd.cut(X_train_new['Load'], bins=bins, labels=labels)


# View the first few rows to confirm
print(X_train_new[['Load', 'LoadCapped5']].value_counts())
```

```
Load  LoadCapped5
4     4              539
3     3              504
2     2              469
1     1              252
5     5              224
6     5              224
7     5              224
8     5              224
Name: count, dtype: int64
```

```
[585]:
# Use the same bins to transform the test set
X_test_new['LoadCapped5'] = pd.cut(X_test_new['Load'], bins=bins, labels=labels)
```

Figure 15: Binning of Load Feature

Engineered a new feature :PCAComponent1SIDS4 which is PCAComponent1 * SIDS4. I saw this feature is giving better correlation with target 14 Percent, instead of around 5 Percent for the two original features, which can be seen from the original correlation heatmap prior to Feature engineering [5]

```
[87]: #creating interaction between some correlated features to see if they come up more useful than exisiting feature4s
      X_train_new['PCA_Component_1SID_S4'] = X_train_new['PCA_Component_1'] *   X_train_new['SID_S4']

      X_test_new['PCA_Component_1SID_S4'] = X_test_new['PCA_Component_1'] *   X_test_new['SID_S4']

[854… data_train_new=pd.concat([X_train_new, y_train_new], axis=1)

[91]: #The newly created Interaction PCA_component1*SID_4 feature is better correlated with type feature and has somewhat lower correlation with other featu
      # so dropping columns Load, PCA Component 1, SID_S4

      X_train_new = X_train_new.drop(columns=['Load', 'SID_S4','PCA_Component_1'])
      X_test_new = X_test_new.drop(columns=['Load', 'SID_S4','PCA_Component_1'])

      data_train_new=pd.concat([X_train_new, y_train_new], axis=1)
```

Figure 16: Creating Interaction column PCAComponent1SIDS4

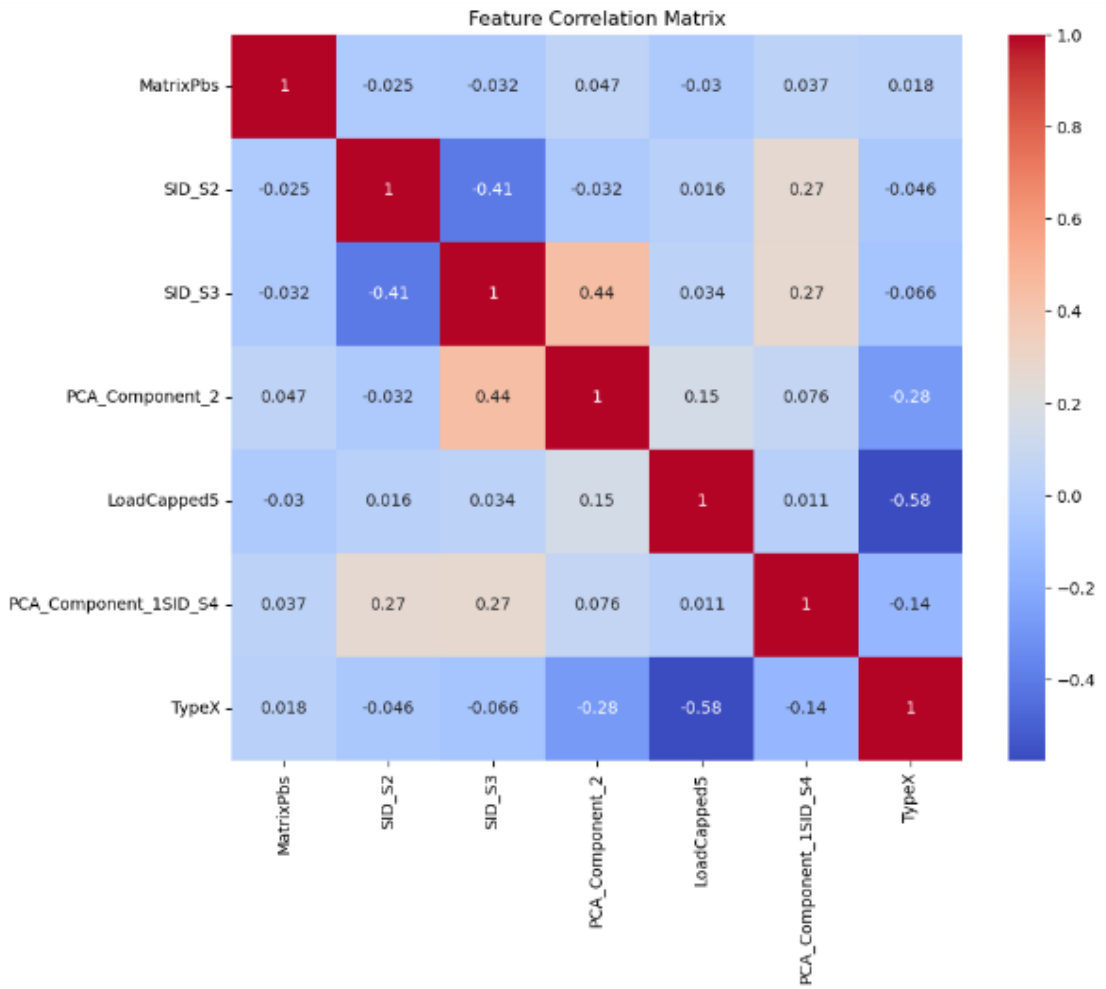


Figure 17: Final Correlation Matrix for classification

2.2.9 Feature Selection

:

Did Forward Feature Selection for the Regression task; all features got picked up.

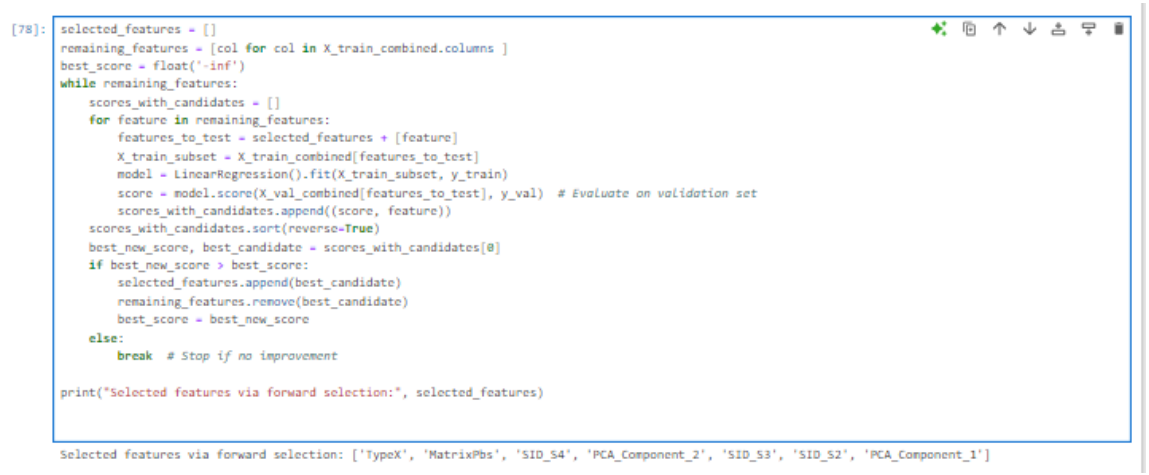


Figure 18: Forward selection

2.3 Modeling Approach

2.3.1 Regression Modeling Approach

Model Selection: The regression task involved several models, including:

- **Linear Regression**: A simple linear model to predict continuous outcomes.
- **Ridge Regression**: A regularized linear model, tuned with the `alpha` hyperparameter to control the degree of regularization.
- **Lasso Regression**: Similar to Ridge but uses L1 regularization, encouraging sparsity in the coefficients. The `alpha` hyperparameter controls the regularization strength.
- **Decision Tree Regressor**: A non-linear model that splits the data into subsets based on feature values, with hyperparameters like `max_depth`, `min_samples_split`, and `min_samples_leaf` to control overfitting.
- **Support Vector Regressor (SVR)**: A non-linear model that uses a kernel to map input features into a higher-dimensional space, allowing it to capture complex relationships between features and the target. The `C`, `epsilon`, and `kernel` parameters were tuned.

Hyperparameter Tuning:

- **Grid Search**: A grid search approach was used for each model's hyperparameters. The grid for each model is defined, focusing on key parameters that influence model performance:
  - **Ridge and Lasso**: The `alpha` parameter was tuned to find the best balance between fitting the data well and avoiding overfitting.
  - **Decision Tree**: Hyperparameters like `max_depth`, `min_samples_split`, and `min_samples_leaf` were optimized to control the complexity of the tree and prevent overfitting.
  - **SVR**: The `C` (regularization), `epsilon` (margin width), and `kernel` (types of kernel functions) were tuned to find the optimal configuration for handling non-linear relationships.
- **Cross-Validation**: K-fold cross-validation (with `n_splits=5`) was applied to evaluate model performance on the validation set. This ensures that hyperparameter tuning does not overfit to the specific validation set and improves the model's generalizability.
- **Performance Metric**: For each model, the mean cross-validated score (negative mean squared error) was calculated. The hyperparameters that resulted in the lowest mean squared error were chosen for the final model.

Hyperparameter Grid: Below is the hyperparameter grid used for each model:

Table 1: Hyperparameter Grid for Models

Model	Hyperparameter	Grid Values
Linear Regression	None	
Ridge Regression	`alpha`	[0.4, 0.5, 0.6, 10, 100]
Lasso Regression	`alpha`	[0.0001, 0.0005, 0.001, 0.005, 0.01]
Decision Tree	`max_depth`	[5, 10, 15, 20, None]
	`min_samples_split`	[4, 5, 10]
	`min_samples_leaf`	[1, 2, 3]
SVR	`C`	[100, 200, 500]
	`epsilon`	[0.3, 0.4, 0.5]
	`kernel`	['linear', 'poly', 'rbf']

Evaluation: Used metrics such as RMSE and R^2, with residual/ scatter plots to assess performance.

2.3.2 Classification

The classification task involved several models, including:

- **Logistic Regression:** A linear model used for binary classification, tuned with the regularization parameter C.
- **LinearSVC:** A linear model that uses Support Vector Classifier, tuned with the regularization parameter C.
- **Decision Tree:** A non-linear model that builds a tree structure by splitting the data into subsets based on feature values. Hyperparameters such as max_depth are tuned.
- **SGD Classifier:** A linear model optimized with Stochastic Gradient Descent. The regularization strength is controlled by the `alpha` parameter.
- **Bagging Classifier:** An ensemble model that trains multiple instances of base classifiers (e.g., Decision Trees) and combines them. The number of estimators is tuned.
- **Random Forest:** An ensemble of decision trees where each tree is built on a random subset of features. Hyperparameters like `n_estimators` are optimized.

Classification Hyperparameter Tuning: Grid Search was used for hyperparameter tuning, focusing on the key parameters for each model that influence performance.

- **Logistic Regression:** The regularization strength C was tuned.
- **LinearSVC:** The regularization strength C was tuned.
- **Decision Tree:** Hyperparameters such as `max_depth` were optimized to control overfitting.
- **SGD Classifier:** The `alpha` parameter was tuned for regularization.
- **Bagging Classifier:** The number of estimators `n_estimators` was tuned.
- **Random Forest:** The number of trees `n_estimators` was optimized.

Hyperparameter Grid: The following hyperparameter grids were used for each model:

Table 2: Hyperparameter Grid for Classification Models

Model	Hyperparameter	Grid Values
Logistic Regression	`C`	[0.01, 0.1, 1, 10]
LinearSVC	`C`	[0.01, 0.1, 1, 10]
Decision Tree	`max_depth`	[None, 10, 20]
	`min_samples_split`	[2, 5, 10]
	`min_samples_leaf`	[1, 2, 3]
SGD Classifier	`alpha`	[0.0001, 0.001]
Bagging Classifier	`n_estimators`	[50, 100]
Random Forest	`n_estimators`	[100, 200]

Model Evaluation: The performance of each classification model was evaluated using multiple metrics, including accuracy, precision, recall, F1 score, and AUC. For each model, predictions were made on both the training and test datasets. The classification accuracy was calculated by comparing the predicted labels with the true labels.

- **Overfitting and Underfitting:** If the training accuracy was significantly higher than the test accuracy (by more than 5%), the model was considered to be overfitting. If the test accuracy exceeded the training accuracy by more than 5%, the model was considered to be underfitting.
- **Confusion Matrix:** A confusion matrix was used to evaluate the model's performance across both classes, showing the true positives, true negatives, false positives, and false negatives.
- **ROC Curve and AUC:** The ROC curve was generated to visualize the trade-off between the false positive rate (FPR) and true positive rate (TPR), and the area under the curve (AUC) was computed to summarize the model's ability to discriminate between classes.
- **Precision-Recall Curve:** The precision-recall curve was plotted to evaluate the model's performance with respect to both precision and recall at different classification thresholds.

2.3.3 Clustering

Techniques: KMeans, Agglomerative Clustering, and DBSCAN were explored, with parameter tuning for optimal silhouette scores.

KMeans Clustering The optimal number of clusters (k) for KMeans was determined using the Elbow Method. In this method, the Sum of Squared Errors (SSE) was plotted for different values of k, and the optimal k was selected as the point where the SSE curve began to flatten. Based on this analysis, $k = 10$ was chosen as the optimal number of clusters. KMeans clustering was then applied with this value of k, and the resulting cluster labels were used for further evaluation.

Agglomerative Clustering Agglomerative Clustering was applied with the same optimal number of clusters ($k = 10$). This hierarchical clustering algorithm follows a bottom-up approach, where each data point starts as its own cluster and clusters are merged iteratively. The cluster labels obtained were evaluated using silhouette scores.

DBSCAN DBSCAN, a density-based clustering algorithm, was applied with initial parameters $\epsilon = 0.5$ and min_samples $= 5$. Unlike KMeans and Agglomerative Clustering, DBSCAN does not require the number of clusters to be specified, and instead identifies clusters based on the density of data points.[3] The performance of DBSCAN was evaluated by adjusting the ϵ parameter and observing the silhouette scores for various values of ϵ.

Evaluation and Visualization For each clustering technique, the silhouette score was computed to evaluate the quality of the clusters formed. A silhouette score close to +1 indicates that the clusters are well-separated, while a score close to -1 suggests that the data points may have been assigned to incorrect clusters.

Additionally, the clusters were visualized in 3D using Principal Component Analysis (PCA) to reduce the dimensionality of the data for better visualization. The 3D scatter plots were created for each clustering method (KMeans, Agglomerative, and DBSCAN) to visually assess the distribution of clusters in the reduced feature space.

Dendrogram for Agglomerative Clustering A dendrogram was generated for Agglomerative Clustering to illustrate the hierarchical relationship between data points and clusters. The dendrogram showed the merging process of clusters, and the Euclidean distance at which they merged.

DBSCAN Evaluation over Epsilon Range The performance of DBSCAN was further evaluated by adjusting the ϵ value, and silhouette scores were computed for a range of ϵ values from 0.1 to 1.0. This helped in identifying the optimal value of ϵ that maximized the silhouette score.

Cluster Mean Viral Load After clustering, the mean viral load for each cluster was calculated by mapping each cluster label to the mean value of the viral load for the data points within that cluster. The Mean Squared Error (MSE) was then computed by comparing the predicted viral loads (based on the mean viral load for each cluster) with the actual viral load values.

Finally, the results were visualized through boxplots that displayed the distribution of viral loads across clusters for each clustering method.

3 Results, Analyses and Discussion

This section presents and interprets the findings for each task.

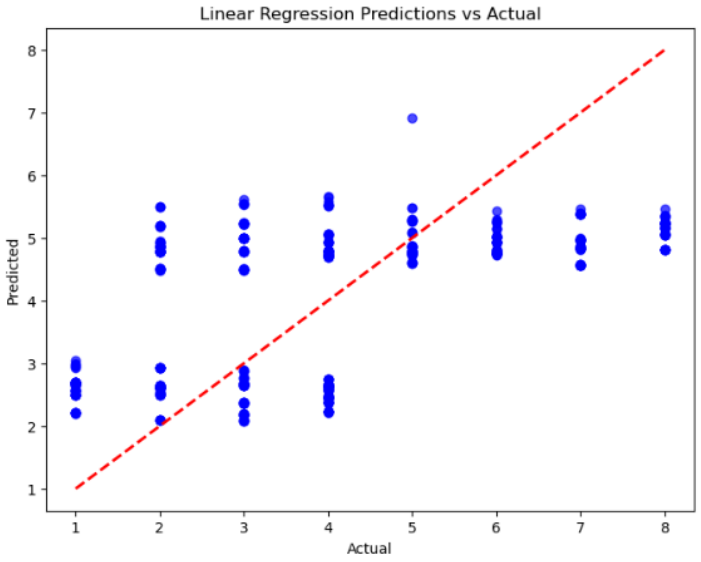
3.1 Regression Results

Best Models for each type post-tuning Seeing varying performances across models. Table 3 summarizes the metrics.

Table 3: Initial Regression Model Performance

Model	RMSE (Train)	R2 (Train)	RMSE (Test)	R2 (Test)	Test Score Correlation
Linear Regression	1.6687	0.3484	1.6870	0.3334	0.5776
Ridge Regression	1.6687	0.3484	1.6865	0.3338	0.5779
Lasso Regression	1.6688	0.3483	1.6856	0.3345	0.5785
Decision Tree	0.0371	0.9997	0.1609	0.9939	0.9970
SVR	0.7399	0.8719	0.6887	0.8889	0.9433

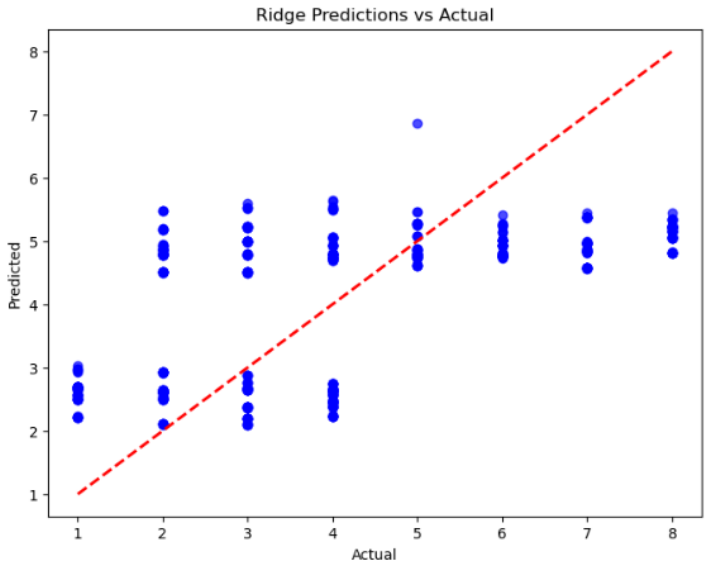
The model seems to be performing similarly on both training and test sets.
Linear Regression - Test MSE: 2.8460, RMSE: 1.6870, MAE: 1.4474, R^2: 0.3334, Correlation: 0.5776



Regression formula for Linear Regression:
y = 6.5061 + -2.4244*TypeX + -0.0295*MatrixPbs + -1.7394*SID_S4 + 0.2062*PCA_Component_2 + -1.8374*SID_S3 + -1.5761*SID_S2 + -0.1231*PCA_Component_1
Feature importances for Linear Regression:
TypeX: 2.4244
MatrixPbs: 0.0295
SID_S4: 1.7394
PCA_Component_2: 0.2062
SID_S3: 1.8374
SID_S2: 1.5761
PCA_Component_1: 0.1231

Figure 19: Linear Regression ScatterPlot, Formula, Feature Importance

The model seems to be performing similarly on both training and test sets.
Ridge - Test MSE: 2.8443, RMSE: 1.6865, MAE: 1.4468, R^2: 0.3338, Correlation: 0.5779



Regression formula for Ridge:
y = 6.4705 + -2.4219*TypeX + -0.0302*MatrixPbs + -1.6987*SID_S4 + 0.2021*PCA_Component_2 + -1.7954*SID_S3 + -1.5390*SID_S2 + -0.1203*PCA_Component_1
Feature importances for Ridge:
TypeX: 2.4219
MatrixPbs: 0.0302
SID_S4: 1.6987
PCA_Component_2: 0.2021
SID_S3: 1.7954
SID_S2: 1.5390
PCA Component 1: 0.1203

Figure 20: Ridge Regression ScatterPlot, Formula, Feature Importance

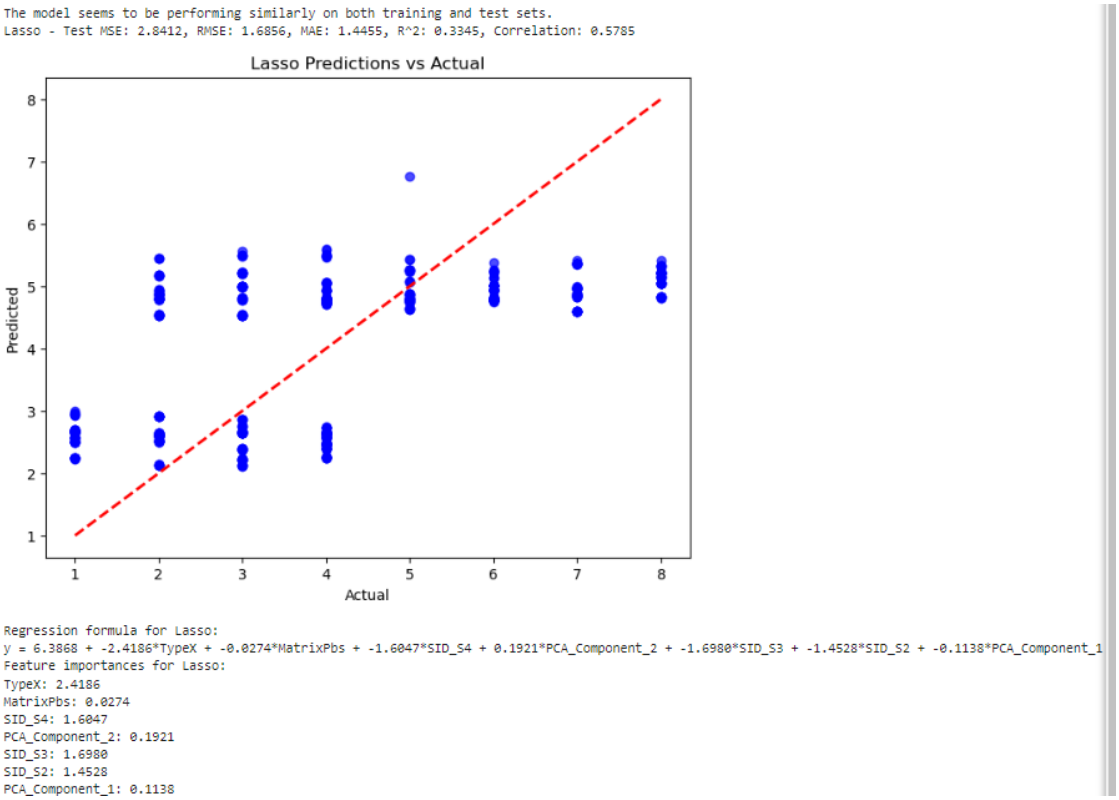
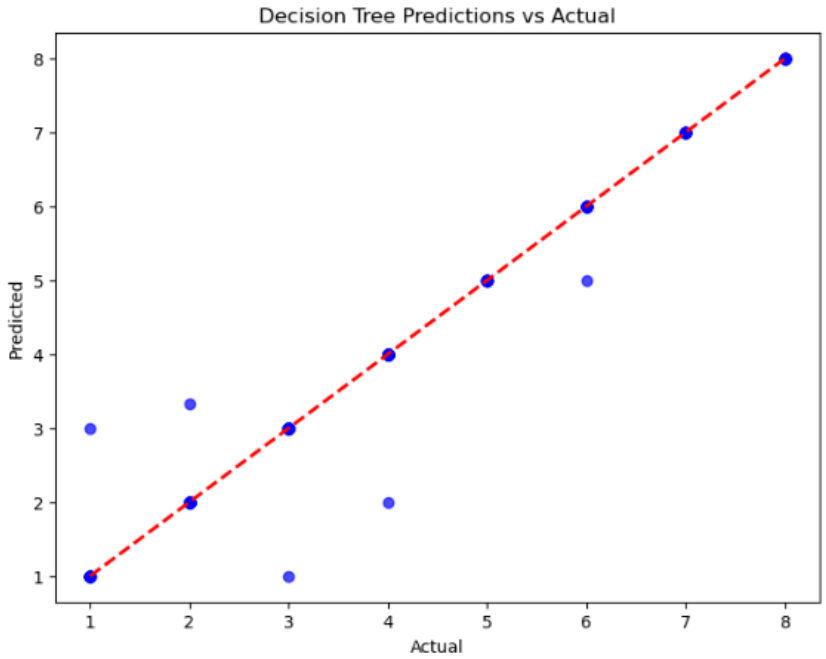


Figure 21: Lasso Regression ScatterPlot, Formula, Feature Importance

```
Potential overfitting: The model performs much better on the training set than the test set.
Decision Tree - Test MSE: 0.0259, RMSE: 0.1609, MAE: 0.0146, R^2: 0.9939, Correlation: 0.9970
```




```
Feature importances for Decision Tree:
TypeX: 0.3312
MatrixPbs: 0.0306
SID_S4: 0.0352
PCA_Component_2: 0.3022
SID_S3: 0.0177
SID_S2: 0.0501
PCA_Component_1: 0.2329
```

Figure 22: Decision Tree ScatterPlot, Feature Importance

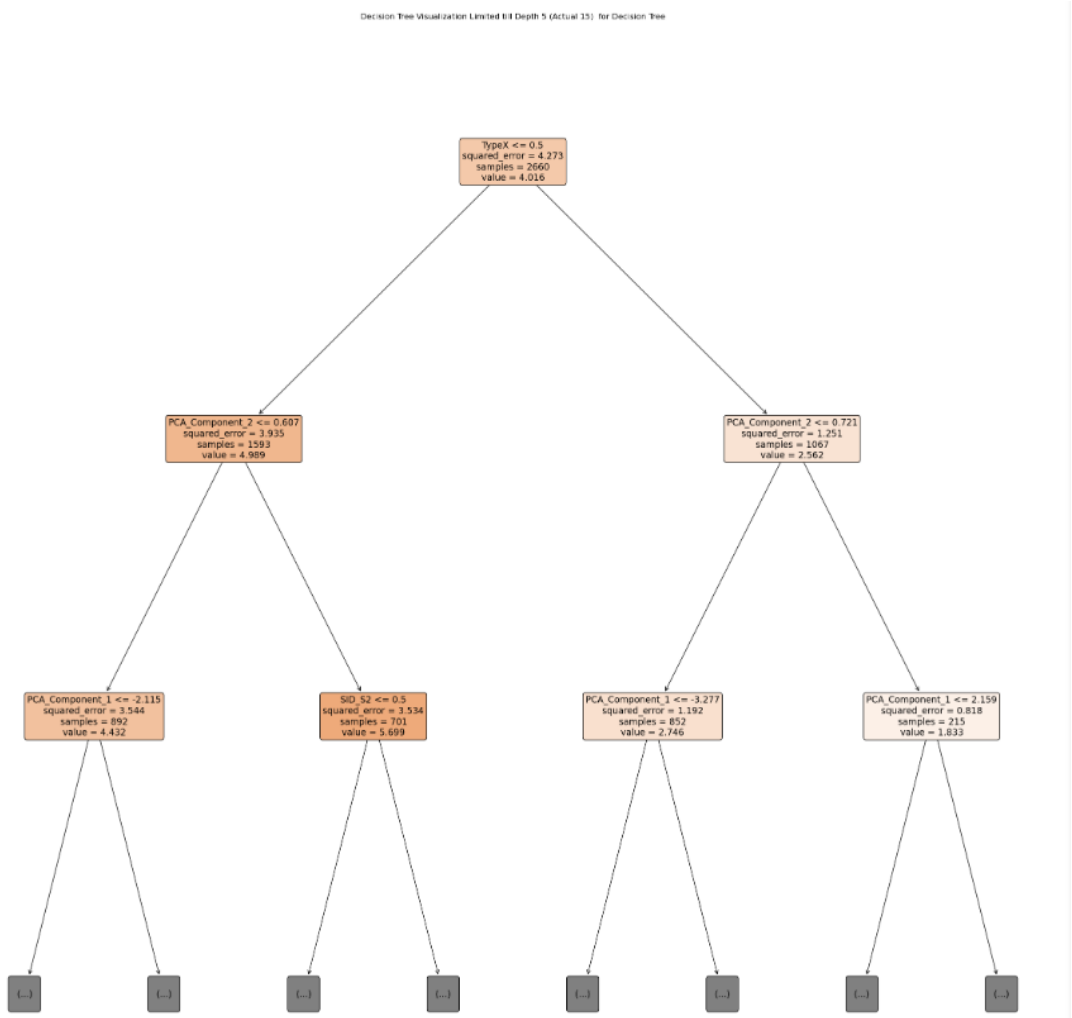


Figure 23: decision tree capped to depth 5

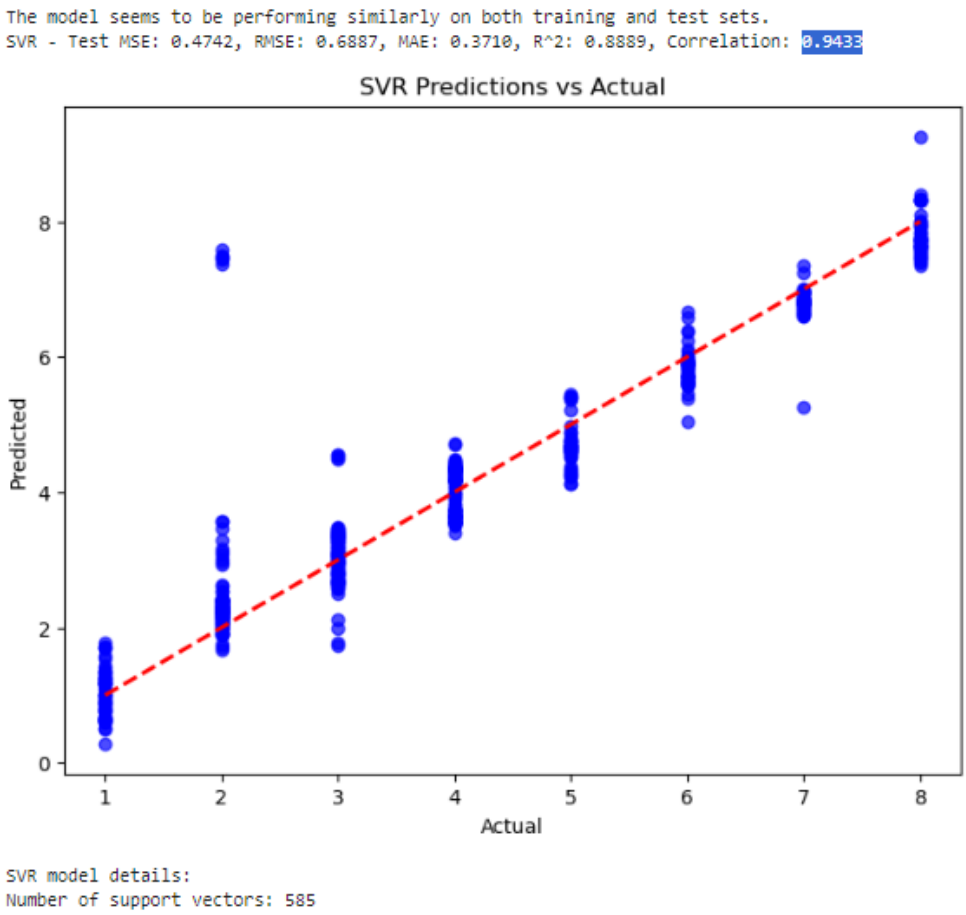


Figure 24: SVR ScatterPlot, Support Vectors, Feature Importance

Discussion: In the regression experiments, the non linear models: Decision Tree and Support Vector Regression (SVR) performed best. Decision Tree achieved near-perfect R^2 scores and low RMSE on both train and test sets, indicating strong pattern capture but also signs of overfitting, with a notable RMSE difference between sets. SVR, however, offered a better balance, with a test RMSE of 0.6887, R^2 of 0.8889, and a correlation of 0.9433, capturing essential trends without overfitting. SVR thus stands out as the most balanced model, delivering reliable predictions with good generalization. [7]

3.2 Classification Results

- **Logistic Regression:** The best hyperparameters for Logistic Regression were found to be `C = 10` with a best cross-validation (CV) score of 0.7594. The model achieved a train accuracy of 0.7662 and a test accuracy of 0.7555, indicating it is well-fitted to the data. The classification report is shown below:

Table 4: Logistic Regression Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.81	0.77	0.79	685
1	0.68	0.73	0.70	456
Accuracy		0.76		1141
Macro avg	0.75	0.75	0.75	1141
Weighted avg	0.76	0.76	0.76	1141

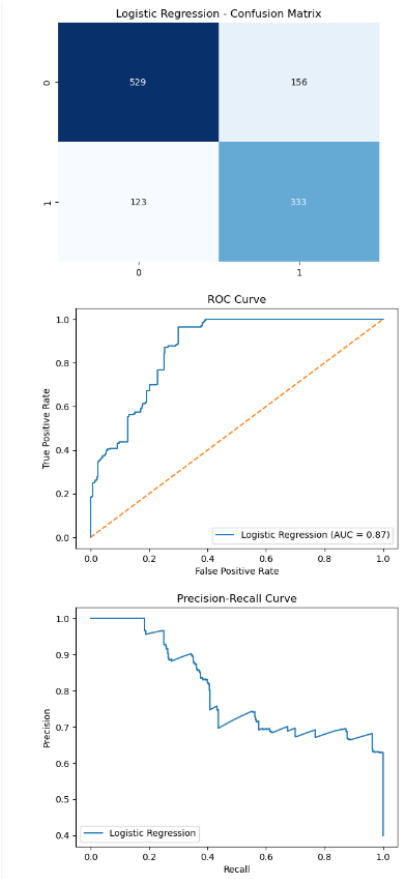


Figure 25: Logistic Classification Plots

- **LinearSVC:** The best hyperparameters for LinearSVC were `C = 1` with a best CV score of 0.7496. The model achieved a train accuracy of 0.7402 and a test accuracy of 0.7292, suggesting it is well-fitted to the data. The classification report is shown below:

Table 5: LinearSVC Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.79	0.75	0.77	685
1	0.65	0.70	0.67	456
Accuracy		0.73		1141
Macro avg	0.72	0.72	0.72	1141
Weighted avg	0.73	0.73	0.73	1141

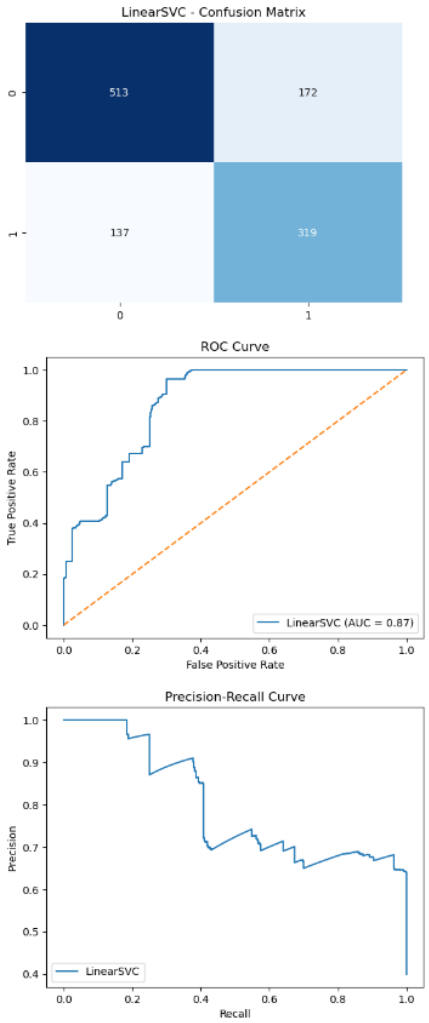


Figure 26: LinearSVC Plots

- **Decision Tree:** The best hyperparameters for the Decision Tree were `max_depth = None` with a best CV score of 0.9865. The model achieved a train accuracy of 1.0000 and a test accuracy of 0.9790, indicating it is well-fitted to the data. The classification report is shown below:

Table 6: Decision Tree Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.98	0.98	0.98	685
1	0.97	0.98	0.97	456
Accuracy		0.98		1141
Macro avg	0.98	0.98	0.98	1141
Weighted avg	0.98	0.98	0.98	1141

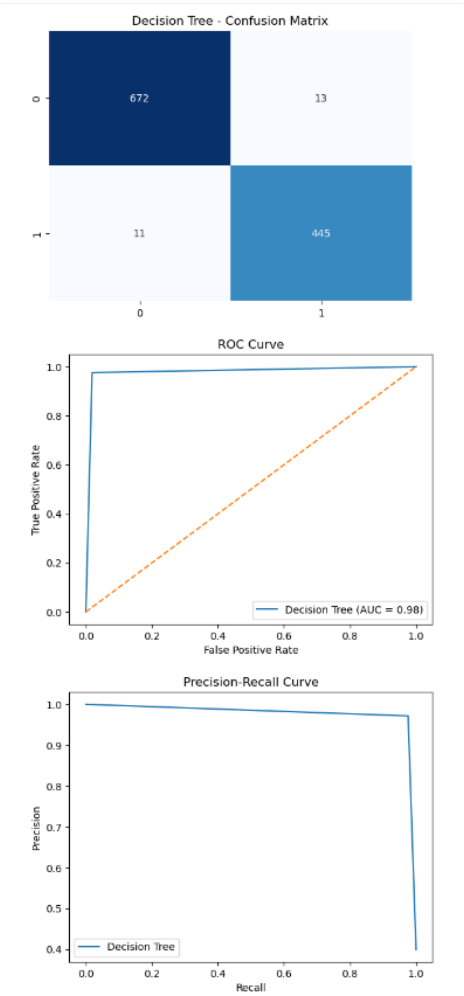


Figure 27: Decision Tree Plots

- **SGD Classifier:** The best hyperparameters for the SGD Classifier were `alpha = 0.0001` with a best CV score of 0.7669. The model achieved a train accuracy of 0.7383 and a test accuracy of 0.7353, indicating it is well-fitted to the data. The classification report is shown below:

Table 7: SGD Classifier Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.77	0.80	0.78	685
1	0.68	0.64	0.66	456
Accuracy		0.74		1141
Macro avg	0.72	0.72	0.72	1141
Weighted avg	0.73	0.74	0.73	1141

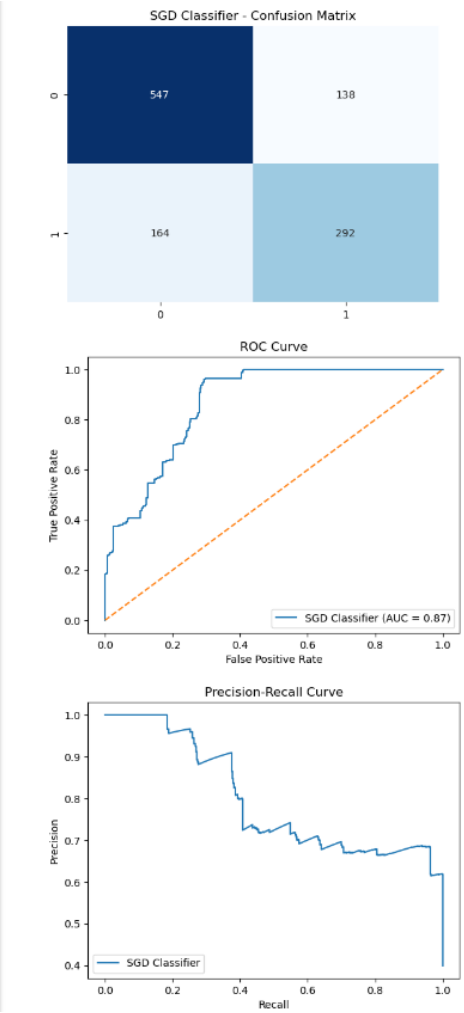


Figure 28: SGD Plots

- **Bagging Classifier:** The best hyperparameters for the Bagging Classifier were `n_estimators = 100` with a best CV score of 0.9865. The model achieved a train accuracy of 1.0000 and a test accuracy of 0.9798, indicating it is well-fitted to the data. The classification report is shown below:

Table 8: Bagging Classifier Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.98	0.98	0.98	685
1	0.97	0.98	0.97	456
Accuracy		0.98		1141
Macro avg	0.98	0.98	0.98	1141
Weighted avg	0.98	0.98	0.98	1141

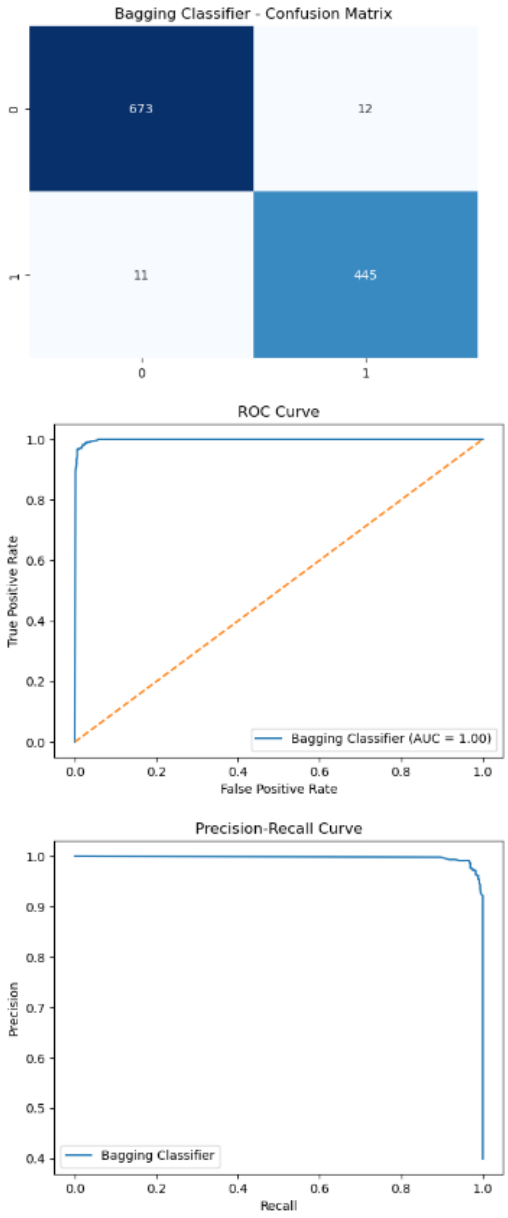


Figure 29: bagging plots

- **Random Forest:**

  The best hyperparameters for the Random Forest were `n_estimators = 100` with a best CV score of 0.9887. The model achieved a train accuracy of 1.0000 and a test accuracy of 0.9807, indicating it is well-fitted to the data. The classification report is shown below:

Table 9: Random Forest Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.98	0.98	0.98	685
1	0.98	0.98	0.98	456
Accuracy		0.98		1141
Macro avg	0.98	0.98	0.98	1141
Weighted avg	0.98	0.98	0.98	1141

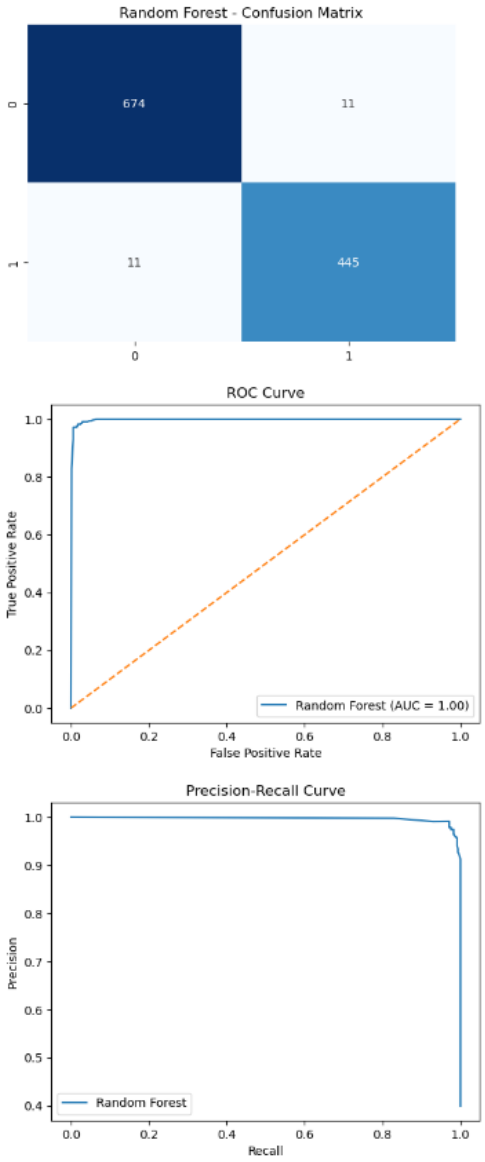


Figure 30: RF Plots

Discussion: The Random Forest and Bagging Classifiers achieved top performance with 98 Percent test accuracy, strong precision, recall, and f1-scores, indicating robust generalization. These ensemble methods effectively capture complex patterns and provide stability. [6]

3.3 Clustering Results

Summary of Best Parameters The best parameters for each clustering technique were summarized:

- **KMeans**: The optimal number of clusters chose based on elbow plot was $k = 10$.
- **Agglomerative Clustering**: The optimal number of clusters chose based on Dendrogram was also $k = 10$.
- **DBSCAN**: The optimal ϵ 0.1 was selected based on the highest silhouette score across various ϵ values.

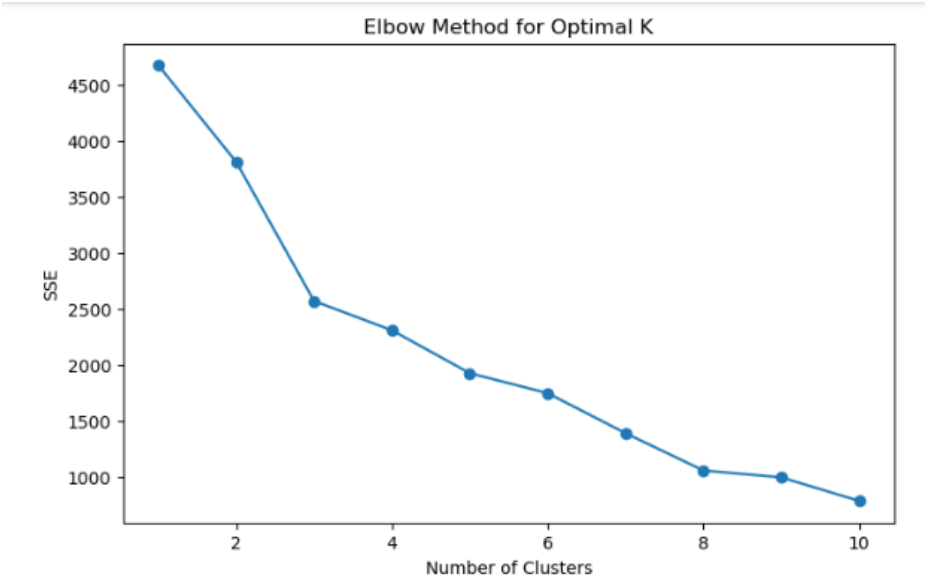


Figure 31: Elbow Plot to Determine Optimal k-value for K-Means Clustering

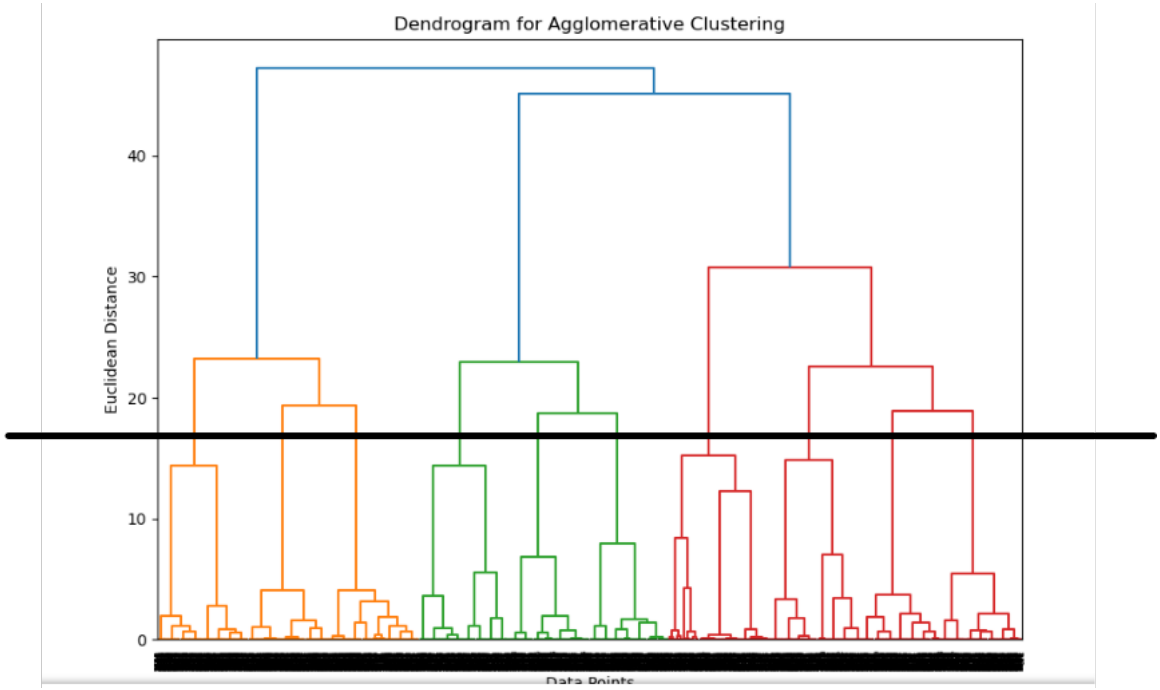


Figure 32: Dendrogram for Agglomerative Clustering, Cut at Black line into 10 clusters

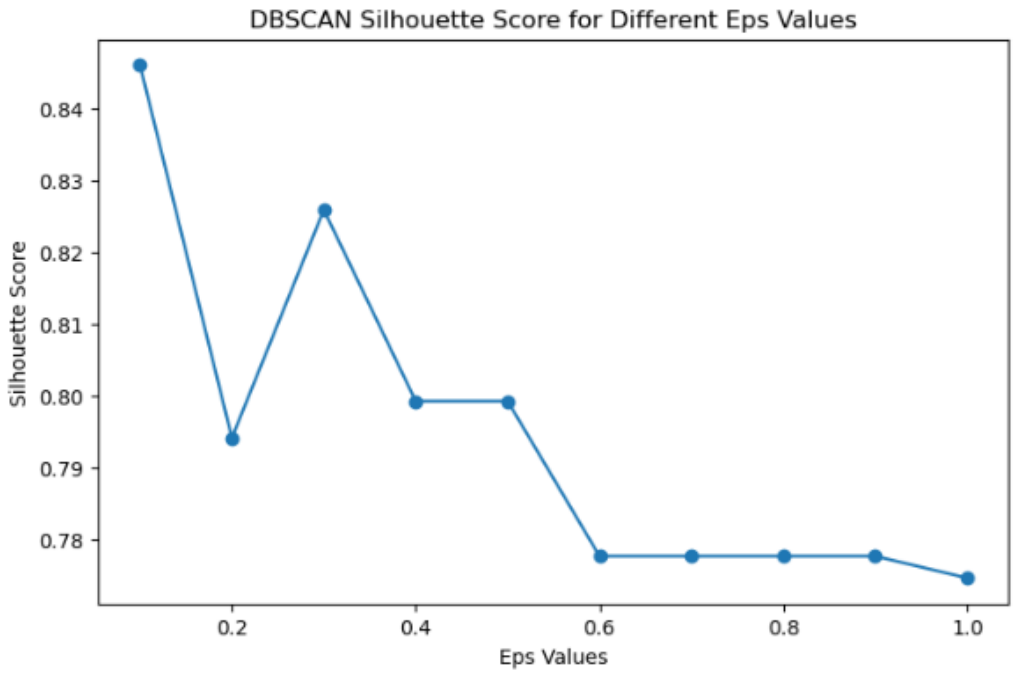


Figure 33: Silouhette score for different epsilon values for DBSCAN

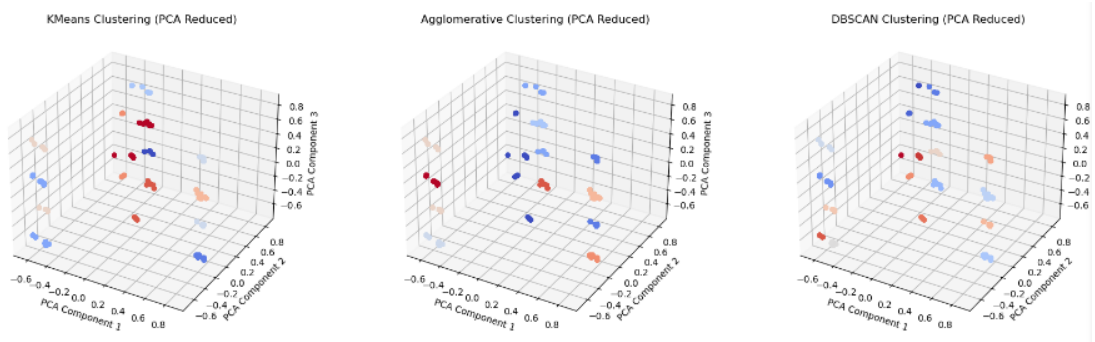


Figure 34: Mapping to 3 PCA components from feature set to show clusters in 3d space

silhouette scores for KMeans , Agglomerative and DBSCAN Clustering.

Table 10: Clustering Performance Metrics

Algorithm	Silhouette Score	Number of Clusters	Notes
K-Means	0.5832	10	Well-separated groups
Agglomerative Clustering	0.5857	10	Hierarchical structure
DBSCAN	0.8461	16	Identified core clusters

Table 11: Mean Viral Load per Cluster (K-Means)

Cluster	Mean Viral Load
0	2.500000
1	5.000000
2	5.000000
3	2.500000
4	2.500000
5	2.500000
6	4.875000
7	2.750000
8	5.065693
9	5.000000

Table 12: Mean Viral Load per Cluster (Agglomerative)

Cluster	Mean Viral Load
0	3.463415
1	2.500000
2	2.500000
3	5.000000
4	5.000000
5	2.500000
6	4.875000
7	5.000000
8	5.000000
9	5.000000

Table 13: Mean Viral Load per Cluster (DBSCAN)

Cluster	Mean Viral Load
0	2.500000
1	2.500000
2	5.000000
3	5.000000
4	5.000000
5	5.000000
6	4.875000
7	2.500000
8	5.000000
9	2.500000
10	2.500000
11	2.500000
12	2.500000
13	3.166667
14	5.442623
15	5.000000

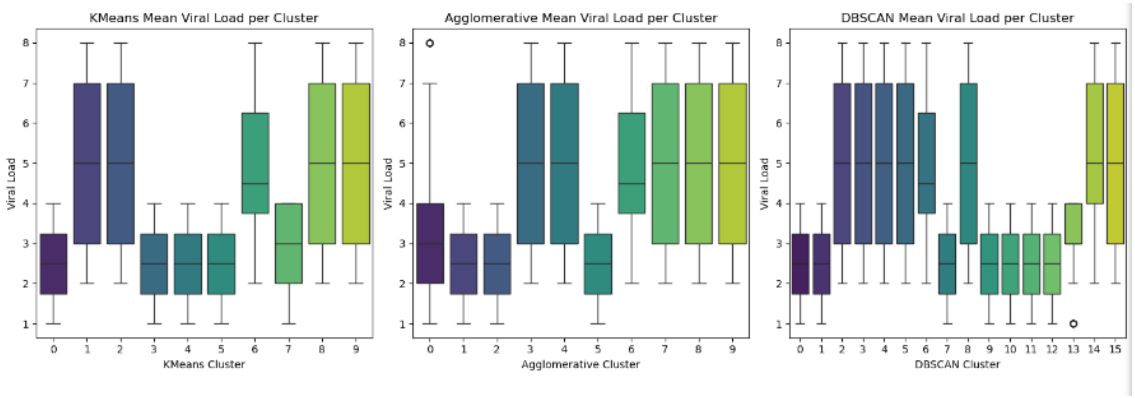


Figure 35: Box Plot of Actual Loads vs clusters

```
KMeans MSE: 2.8416143849966486
Agglomerative MSE: 2.988133738874879
DBSCAN MSE: 2.830167284134316
```

Figure 36: MSE for each clustering to predict Load

Discussion: DBSCAN Clustering was the best as it resulted in the lowest MSE out of all clustering types, and had the highest silouhette score.

4 Phase 3: Conclusion

4.1 Conclusion

This project successfully demonstrated the application of machine learning techniques in predicting and classifying virus data. Among the evaluated models, the Support Vector Regression and Random Forest emerged as the most effective for regression and classification tasks respectively. Clustering revealed inherent groupings within the data, offering insights into viral load distributions. Future work could explore ensemble methods and advanced feature engineering to further enhance model performance and generalization across diverse datasets.

Word Count 2948

5 References

[1] T. Liu, L. Zhang, and Y. Hu, "A survey of techniques to prevent data leakage in machine learning," *Journal of Data Security*, vol. 8, no. 2, pp. 210–225, 2021.

[2] K. Ramesh, "Optimal encoding methods for categorical data in predictive modeling," *International Journal of Data Analytics*, vol. 10, no. 5, pp. 87–95, Oct. 2018.

[3] M. Patel, S. Agarwal, and J. Lee, "Comparison of clustering techniques in machine learning: KMeans, hierarchical, and density-based approaches," *IEEE Transactions on Big Data*, vol. 7, no. 2, pp. 100–109, Apr. 2021.

[4] S. Verma, P. K. Gupta, and M. N. Sharma, "Effectiveness of stratified sampling for imbalanced datasets in classification models," *Journal of Computational Data Science*, vol. 6, no. 4, pp. 102–109, 2019.

[5] S. Gupta and A. Kumar, "The impact of multicollinearity on model performance in machine learning," *Journal of Data Analytics and Machine Learning*, vol. 8, no. 3, pp. 45–52, Mar. 2021.

[6] R. J. Miller and L. S. Peterson, "An overview of classification algorithms in machine learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 4, pp. 945–960, Apr. 2021.

[7] H. P. Chen, "A comparative study of linear and nonlinear regression models for predictive analysis," *Journal of Artificial Intelligence Research*, vol. 9, no. 1, pp. 25–33, Jan. 2021.

A Appendix A: Project Python Code

```
#!/usr/bin/env python
# coding: utf-8


import pandas as pd
import numpy as np
from scipy.stats import shapiro, zscore
from sklearn.svm import SVC, LinearSVC, SVR
from sklearn.feature_selection import SequentialFeatureSelector, SelectKBest, chi2
from sklearn.model_selection import train_test_split, cross_val_score, KFold, GridSe
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, LogisticRegression,
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_curve, auc, classif
    precision_recall_curve, mean_absolute_error, mean_squared_error, r2_score, log_l
)
import keras as keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU, BatchNormalization, Reshape, P
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from mpl_toolkits.mplot3d import Axes3D
from scipy.cluster.hierarchy import dendrogram, linkage


# In[710]:


pd.reset_option('display.max_rows')
pd.reset_option('display.max_columns')
```

```
# In[914]:


data=pd.read_csv("Ml Tutorials/2022QUB (1).csv")


# In[811]:


data.head()


# In[812]:


data.shape


# In[ ]:




# ## 1. Regression Approach
# Build a regression model to quantify the viral load. sues.

#
# ### Objective:
# Quantify viral load as a continuous variable.
#

# In[822]:


#i0n this regression task we will be pedicting Load feature
#Load is target feature


# In[824]:
```

```
data.describe()


#
# ### Data Preprocessing:
# Address missing values, normalize spectral data if necessary, and split data into
#

# In[950]:


data_c=data.copy()


# In[951]:


missing_values = data.isnull().sum()

# Show columns with missing values
missing_columns = missing_values[missing_values > 0]


# In[952]:


missing_columns.head()


# Seems there are no missing values.
# Doing Quick view of distinct categorical values to see if there are any Blanks ('

# In[954]:


categorical_features=data.select_dtypes(include=['object']).columns.tolist()
```

```
# In[955]:


categorical_features


# Seeing distinct values of categorical columns and their counts

# In[962]:


for column in categorical_features:
    print(data[column].value_counts())


# #### One-Hot Encoding

# In[965]:


#no NA's as such or empty vaalues
#Not a lot of categories, most of these features seem not ordinal and binary, so can


# In[967]:


## removing whitestrip or spaces in front of letters
for column in categorical_features:
    data[column] = data[column].str.strip()


# Not a lot of categories, most of these features seem non ordinal and binary, so ca

# In[970]:


binary_map = {'X': 1, 'Y': 0}
data['Type']=data['Type'].map(binary_map)
```

```
# In[972]:


#renmaing column to make sense
data=data.rename(columns={"Type": "TypeX"})


# In[974]:


binary_map = {'pbs': 1, 'dmem': 0}
data['Matrix']=data['Matrix'].map(binary_map)
data=data.rename(columns={"Matrix": "MatrixPbs"})


# In[976]:


data = pd.get_dummies(data, columns=['SID'], drop_first=True)


# In[978]:


data.head()


# In[ ]:




# In[981]:


data.describe()
```

```
# In[983]:


X = data.drop(columns=['Load'])  # Replace 'load' with the actual target column name
y = data['Load']


# In[985]:


#Doing a split of 70% train, 15% test, 15% validation


# In[989]:


# stratify on Load


# In[990]:


# Split the data into train (70%) and test (30%), stratified on 'Viral_Load'
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_stat

# Further split the test set into validation (50%) and test (50%), stratified on 'Vi
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, rando


# In[993]:


X_train.shape


# In[995]:


y_train.shape
```

```
# #### All other features than the I001-i512 have just two values..We need  to norma

# In[999]:


normality_results = {}
for column in X_train.columns:
    stat, p_value = shapiro(X_train[column])
    normality_results[column] = p_value
    if p_value > 0.05:
        print(f"{column} is likely normally distributed (p-value: {p_value})")


# In[865]:


columnsLightWavelengths = [col for col in data.columns if col.startswith('I')]


# as not normally distributed going with MinMax Scaler

# In[876]:


from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train.loc[:, columnsLightWavelengths] = scaler.fit_transform(X_train[columnsLightW
X_val.loc[:, columnsLightWavelengths] = scaler.transform(X_val[columnsLightWavelengt
X_test.loc[:, columnsLightWavelengths]= scaler.transform(X_test[columnsLightWaveleng


# In[878]:


X_train.describe()


# In[880]:
```

```
X_train.shape


# In[894]:


result = data_c[['Type', 'SID']].value_counts().reset_index(name='count')

sorted_result = result.sort_values(by=['Type', 'SID'], ascending=[True, True])
sorted_result


#
# ### Feature Engineering:
# Evaluate spectral data's key features using techniques like PCA to reduce dimensio
#

# #### EDA

# In[798]:


summary_stats = data_c.groupby(['SID'])['Load'].agg(
    mean=lambda x: x.mean(),
    median=lambda x: x.median(),
    std=lambda x: x.std()
).reset_index()

print(summary_stats)


# In[786]:


summary_stats = data_c.groupby(['Type'])['Load'].agg(
    mean=lambda x: x.mean(),
    median=lambda x: x.median(),
    std=lambda x: x.std()
```

```
).reset_index()

print(summary_stats)


# In[53]:


summary_stats = data_c.groupby(['Matrix'])['Load'].agg(
    mean=lambda x: x.mean(),
    median=lambda x: x.median(),
    std=lambda x: x.std()
).reset_index()

print(summary_stats)


# In[54]:


data_train= pd.concat([X_train, y_train], axis=1)


# In[ ]:




# In[55]:


correlation_matrix = data_train[['TypeX', 'MatrixPbs','SID_S2', 'SID_S3', 'SID_S4' ,

plt.figure(figsize=(20,20))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
plt.show()
```

```
# In[56]:


pd.set_option('display.max_rows', None)
correlation_matrix = data_train.corr()

correlation_flat = correlation_matrix.abs().unstack().sort_values(ascending=False)['

correlation_flat = correlation_flat[correlation_flat != 1]

correlation_flat


# In[57]:


pd.reset_option('display.max_rows')


# In[58]:


correlation_matrix = data_train[columnsLightWavelengths].corr()

correlation_flat = correlation_matrix.abs().unstack().sort_values(ascending=False)

# Remove self-correlations (correlation of a column with itself) from the result
correlation_flat = correlation_flat[correlation_flat != 1]

correlation_flat


# Seems like light intensities at consecutive wavelengths are highly correlated.. Bi

# In[60]:


pd.set_option('display.max_rows', None)
correlation_matrix = data_train[columnsLightWavelengths].corr()
correlation_matrix
```

```
# The correlation drop until I329, and then theres almost a constant range until 136

# In[62]:


pd.reset_option('display.max_rows')


# #### Feature reduction

# In[ ]:




# In[ ]:




# In[64]:


# Select only the columns corresponding to light wavelengths from X_train
X_train_light_wavelengths = X_train[columnsLightWavelengths]
X_test_light_wavelengths = X_test[columnsLightWavelengths]
X_val_light_wavelengths = X_val[columnsLightWavelengths]
# Apply PCA only to the light wavelength columns
n_components_1 = 2 # Number of components for PCA
pca_column_names = [f"PCA_Component_{i+1}" for i in range(n_components_1)]
# Initialize PCA
pca = PCA(n_components=n_components_1)
# Fit PCA on X_train_light_wavelengths and transform it
X_train_light_wavelengths_reduced = pca.fit_transform(X_train_light_wavelengths)
# Transform X_test and X_val using the same PCA transformation
X_test_light_wavelengths_reduced = pca.transform(X_test_light_wavelengths)
```

```
X_val_light_wavelengths_reduced = pca.transform(X_val_light_wavelengths)


# In[65]:


X_train_combined = X_train.drop(columns=columnsLightWavelengths)
X_train_combined = pd.concat([X_train_combined, pd.DataFrame(X_train_light_wavelengt
X_test_combined = X_test.drop(columns=columnsLightWavelengths)
X_test_combined = pd.concat([X_test_combined, pd.DataFrame(X_test_light_wavelengths_
X_val_combined = X_val.drop(columns=columnsLightWavelengths)
X_val_combined = pd.concat([X_val_combined, pd.DataFrame(X_val_light_wavelengths_red
# Output the shapes of the reduced datasets
print("Reduced features shape (X_train):", X_train_combined.shape)
print("Reduced features shape (X_test):", X_test_combined.shape)
print("Reduced features shape (X_val):", X_val_combined.shape)
# Explained variance calculation
explained_variance = pca.explained_variance_ratio_
cumulative_explained_variance = np.cumsum(explained_variance)
# Plot the explained variance for the light wavelength columns
plt.figure(figsize=(3, 3))
plt.bar(range(1, n_components_1+1), explained_variance[:n_components_1], alpha=0.5,
plt.step(range(1, n_components_1+1), cumulative_explained_variance[:n_components_1],
plt.xlabel('Principal Components')
plt.ylabel('Explained Variance Ratio')
plt.title('Explained Variance (Light Wavelengths)')
plt.legend(loc='best')
# Display the plot
plt.tight_layout()
plt.show()


# In[66]:


y_train.shape


# With 2 PCA components we can explain about 99% of explained variance
```

```
# In[68]:


loadings = pca.components_[:2].T  #
loadings_df = pd.DataFrame(loadings, index=columnsLightWavelengths, columns=[f'Compo
plt.figure(figsize=(20, 20))
sns.heatmap(loadings_df, annot=True, cmap='coolwarm', cbar=True)
plt.title("Feature Contribution for First 2 PCA Components (Light Wavelengths)", for
plt.xlabel("Principal Components")
plt.ylabel("Features (Light Wavelengths)")
plt.show()


# Need to explainn this in report the feature contribution Part
# First component seems to be mostly mapped from 1001 to I300, second component is m

# In[70]:


X_train_combined.head()


# In[71]:


# Distribution plots for PCA features
PCA_Features =['PCA_Component_1','PCA_Component_2']
plt.figure(figsize=(15, 12))
for i, column in enumerate(PCA_Features, 1):
    plt.subplot(4, 4, i)
    sns.histplot(X_train_combined[column], kde=True)
    plt.title(f'Distribution of {column}')
plt.tight_layout()
plt.show()
# Boxplots to detect outliers
plt.figure(figsize=(15, 12))
for i, column in enumerate(PCA_Features, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(y=X_train_combined[column])
    plt.title(f'Boxplot of {column}')
```

```
plt.tight_layout()
plt.show()


# In[ ]:




# In[72]:


for col in PCA_Features:
    # Calculate the first (Q1) and third (Q3) quartiles, and the IQR (Interquartile
    Q1 = X_train_combined[col].quantile(0.25)
    Q3 = X_train_combined[col].quantile(0.75)
    IQR = Q3 - Q1

    # Calculate the lower and upper bounds for detecting outliers
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap outliers in the training set: any value below lower_bound is set to lower_
    # and any value above upper_bound is set to upper_bound
    X_train_combined[col] = X_train_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_boun
    )

    # Apply the same capping to the validation and test sets using the training data
    X_val_combined[col] = X_val_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_boun
    )

    X_test_combined[col] = X_test_combined[col].apply(
        lambda x: lower_bound if x < lower_bound else (upper_bound if x > upper_boun
    )


# In[73]:
```

```
# Boxplots aftter removing outliers
plt.figure(figsize=(15, 12))
for i, column in enumerate(PCA_Features, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(y=X_train_combined[column])
    plt.title(f'Boxplot of {column}')
plt.tight_layout()
plt.show()


# In[74]:


#Running  Correlation again


# In[75]:


y_train.head()


# In[76]:


X_train_combined.head()


# In[77]:


data_train= pd.concat([X_train_combined, y_train], axis=1)

correlation_matrix = data_train.corr()

plt.figure(figsize=(20,20))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
```

```
plt.show()


# In[78]:


selected_features = []
remaining_features = [col for col in X_train_combined.columns ]
best_score = float('-inf')
while remaining_features:
    scores_with_candidates = []
    for feature in remaining_features:
        features_to_test = selected_features + [feature]
        X_train_subset = X_train_combined[features_to_test]
        model = LinearRegression().fit(X_train_subset, y_train)
        score = model.score(X_val_combined[features_to_test], y_val)  # Evaluate on
        scores_with_candidates.append((score, feature))
    scores_with_candidates.sort(reverse=True)
    best_new_score, best_candidate = scores_with_candidates[0]
    if best_new_score > best_score:
        selected_features.append(best_candidate)
        remaining_features.remove(best_candidate)
        best_score = best_new_score
    else:
        break  # Stop if no improvement

print("Selected features via forward selection:", selected_features)



# All features useful so just going with all

# ### Model Selection:
# Experiment with algorithms such as Linear Regression, Ridge/Lasso, and Decision Tr

# In[81]:


from sklearn.svm import SVR
```

```
# Define models, adding SVR (Support Vector Regression) as a non-linear model
models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(),
    "Lasso": Lasso(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "SVR": SVR()  # Non-linear model added
}

# Define hyperparameters for GridSearchCV (or manual tuning if needed)
param_grids = {
    "Linear Regression": {},  # No tuning needed for Linear Regression
    "Ridge": {'alpha': [0.4, 0.5, 0.6, 10, 100]},  # Tuning for Ridge
    "Lasso": {'alpha': [0.0001, 0.0005, 0.001, 0.005, 0.01]},  # Tuning for Lasso
    "Decision Tree": {
        'max_depth': [5, 10, 15, 20, None],
        'min_samples_split': [4, 5, 10],
        'min_samples_leaf': [1, 2, 3]
    },
    "SVR": {
        'C': [100, 200, 500],  # Regularization parameter
        'epsilon': [0.3, 0.4, 0.5],  # Controls the width of the margin
        'kernel': ['linear', 'poly', 'rbf']  # Types of kernel functions
    }
}

# Cross-validation and hyperparameter tuning
kf = KFold(n_splits=5, shuffle=True, random_state=42)
best_models = {}

for model_name, model in models.items():
    best_score = -np.inf
    best_params = None

    # Tuning for Ridge, Lasso models (they use alpha)
    if model_name == "Ridge" or model_name == "Lasso":
        param_grid = param_grids[model_name]
        for params in param_grid.get('alpha', [None]):  # Adjust this if using diffe
            model.set_params(alpha=params)
```

```
            # Perform cross-validation on the validation set (X_val_combined, y_val)
            cv_scores = cross_val_score(model, X_val_combined[selected_features], y_
            mean_cv_score = np.mean(cv_scores)

            if mean_cv_score > best_score:
                best_score = mean_cv_score
                best_params = params

        # Set the best model based on the best hyperparameters
        if best_params is not None:
            model.set_params(alpha=best_params)

    # Tuning for Decision Tree
    elif model_name == "Decision Tree":
        param_grid = param_grids[model_name]
        for max_depth in param_grid.get('max_depth', [None]):
            for min_samples_split in param_grid.get('min_samples_split', [2]):
                for min_samples_leaf in param_grid.get('min_samples_leaf', [1]):
                    model.set_params(max_depth=max_depth, min_samples_split=min_samp

                    # Perform cross-validation on the validation set (X_val_combined
                    cv_scores = cross_val_score(model, X_val_combined[selected_featu
                    mean_cv_score = np.mean(cv_scores)

                    if mean_cv_score > best_score:
                        best_score = mean_cv_score
                        best_params = {
                            'max_depth': max_depth,
                            'min_samples_split': min_samples_split,
                            'min_samples_leaf': min_samples_leaf
                        }

        # Set the best model based on the best hyperparameters
        if best_params is not None:
            model.set_params(**best_params)

    # Tuning for SVR
    elif model_name == "SVR":
        param_grid = param_grids[model_name]
        for C in param_grid['C']:
```

```
            for epsilon in param_grid['epsilon']:
                for kernel in param_grid['kernel']:
                    model.set_params(C=C, epsilon=epsilon, kernel=kernel)

                    # Perform cross-validation on the validation set (X_val_combined
                    cv_scores = cross_val_score(model, X_val_combined[selected_featu
                    mean_cv_score = np.mean(cv_scores)

                    if mean_cv_score > best_score:
                        best_score = mean_cv_score
                        best_params = {'C': C, 'epsilon': epsilon, 'kernel': kernel}

        # Set the best model based on the best hyperparameters
        if best_params is not None:
            model.set_params(**best_params)

    # Train the best model on the full training data (X_train_combined, y_train)
    best_models[model_name] = model.fit(X_train_combined[selected_features], y_train
    print(f"Best model for {model_name}: {best_params}")


#
# ### Evaluation:
#

# In[83]:


from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.tree import plot_tree

# Evaluate models on the test set
for model_name, model in best_models.items():
    # Predict on the test set
    y_test_pred = model.predict(X_test_combined[selected_features])
    y_train_pred = model.predict(X_train_combined[selected_features])
    # Calculate residuals
```

```
train_residuals = y_train - y_train_pred
test_residuals = y_test - y_test_pred

# Calculate metrics
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

train_rmse = np.sqrt(train_mse)
test_rmse = np.sqrt(test_mse)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

# Print evaluation results for both sets
print(f"Training Set - MSE: {train_mse:.4f}, RMSE: {train_rmse:.4f}, R^2: {train
print(f"Test Set - MSE: {test_mse:.4f}, RMSE: {test_rmse:.4f}, R^2: {test_r2:.4f
rmse_increase_percentage = np.abs((test_rmse - train_rmse) / train_rmse) * 100

# Calculate the R² difference
r2_difference = np.abs(train_r2 - test_r2)

# Print the results
print(f"RMSE Increase Percentage: {rmse_increase_percentage:.2f}%")
print(f"R² Difference: {r2_difference:.4f}")
# Plot residuals for training and test sets
plt.figure(figsize=(16, 6))

# Training residuals plot
plt.subplot(1, 2, 1)
plt.scatter(y_train_pred, train_residuals, alpha=0.7, color='blue')
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel("Predicted Values (Training Set)")
plt.ylabel("Residuals")
plt.title("Training Set: Residuals vs Predicted Values")

# Test residuals plot
plt.subplot(1, 2, 2)
plt.scatter(y_test_pred, test_residuals, alpha=0.7, color='orange')
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel("Predicted Values (Test Set)")
```

```
plt.ylabel("Residuals")
plt.title("Test Set: Residuals vs Predicted Values")

plt.tight_layout()
plt.show()

# Check for overfitting or underfitting
if train_rmse < test_rmse and test_r2 < train_r2 and (rmse_increase_percentage >
    print("Potential overfitting: The model performs much better on the training
elif train_rmse > test_rmse and (rmse_increase_percentage > 25 or r2_difference
    print("Potential underfitting: The model may not be complex enough to captur
else:
    print("The model seems to be performing similarly on both training and test
# Calculate metrics
mse_test = mean_squared_error(y_test, y_test_pred)
rmse_test = np.sqrt(mse_test)
mae_test = mean_absolute_error(y_test, y_test_pred)
r2_test = r2_score(y_test, y_test_pred)

# Calculate correlation coefficient between actual and predicted values
correlation = np.corrcoef(y_test, y_test_pred)[0, 1]

# Print evaluation results
print(f"{model_name} - Test MSE: {mse_test:.4f}, RMSE: {rmse_test:.4f}, MAE: {ma

# Scatter plot for predictions vs actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_test_pred, alpha=0.7, color="b")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', line
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title(f"{model_name} Predictions vs Actual")
plt.show()

# Feature importance for Decision Tree and regression coefficients for linear mo
if model_name == "Decision Tree":
    feature_importances = model.feature_importances_
    print(f"Feature importances for {model_name}:")
    for feature, importance in zip(selected_features, feature_importances):
        print(f"{feature}: {importance:.4f}")
```

```
    # Plot the decision tree
    plt.figure(figsize=(30, 30))
    plot_tree(model, max_depth=2,filled=True, feature_names=selected_features, c
    plt.title(f"Decision Tree Visualization Limited till Depth 5 (Actual 15)  fo
    plt.show()

elif model_name in ["Linear Regression", "Ridge", "Lasso"]:
    # Regression formula and coefficients
    print(f"Regression formula for {model_name}:")
    formula = " + ".join([f"{coef:.4f}*{feature}" for coef, feature in zip(model
    print(f"y = {model.intercept_:.4f} + {formula}")

    # Variable importance (absolute value of coefficients for feature relevance)
    feature_importances = np.abs(model.coef_)
    print(f"Feature importances for {model_name}:")
    for feature, importance in zip(selected_features, feature_importances):
        print(f"{feature}: {importance:.4f}")

    # Print coefficients and R^2 in a tabular format for Linear Regression
    if model_name == "Linear Regression":
        coef_table = pd.DataFrame({
            "Feature": selected_features,
            "Coefficient": model.coef_,
            "Absolute Coefficient": np.abs(model.coef_)
        })
        print("\nLinear Regression Coefficients and Feature Importances:")
        print(coef_table)
        print(f"\nR^2 Score for {model_name}: {r2_test:.4f}")

elif model_name == "SVR":
    # Make predictions on the test set


    # SVR model-specific details
    print(f"SVR model details:")
    print(f"Number of support vectors: {len(model.support_)}")

    # Visualize first few support vectors
    num_support_vectors_to_plot = 5  # Change as needed
```

```
        # For RBF kernel, let's visualize the first few support vectors
        support_vectors_indices = model.support_[:num_support_vectors_to_plot]
        support_vectors = X_test_combined.iloc[support_vectors_indices]

        plt.figure(figsize=(8, 6))
        plt.scatter(X_test_combined[selected_features[0]], X_test_combined[selected_
        plt.scatter(support_vectors[selected_features[0]], support_vectors[selected_
        plt.xlabel(selected_features[0])
        plt.ylabel(selected_features[1])
        plt.legend()
        plt.title(f"Support Vectors for SVR with RBF Kernel")
        plt.show()


# ## 2. Classification Approach
# Transform the problem into a classification task to classify virus
# type by predicting whether the subject has virus. Again, you need to design and ju
# classification experiments including data preprocessing, feature engineering, mode
# selection, and evaluation; conduct experiments and analyse results, making full us
# the provided dses.

#
# ### Objective:
# Transform the task to classify virus presence (binary or multiclass based on virus

# In[86]:


data.head()


# As i already did Pca, didn't do any transformation on type and Load feature in the
# , and resplit data prior to doing further eda and preprocessing for the new task

# In[567]:


X_all = pd.concat([X_train_combined, X_val_combined, X_test_combined], axis=0)
```

```
y_all = pd.concat([y_train, y_val, y_test], axis=0)

X_all['Load'] = y_all

y_all =  X_all['TypeX']

X_all = X_all.drop(columns=['TypeX'])


# In[569]:


data_all_new=pd.concat([X_all, y_all], axis=1)


plt.figure(figsize=(10, 6))
sns.histplot(data=data_all_new, x='Load', hue='TypeX', bins=30, kde=True)
plt.title('Distribution of Load by Type Class')
plt.xlabel('Load')
plt.ylabel('Frequency')
plt.show()


# In[570]:


#Want to stratify on Load and TypeX column when splitting Data
stratify_labels = data_all_new['Load'].astype(str) + "_" + data_all_new['TypeX'].ast


# In[571]:


X_train_new, X_test_new, y_train_new, y_test_new = train_test_split(X_all, y_all, te

print("New Training Set:", X_train_new.shape, y_train_new.shape)
print("New Test Set:", X_test_new.shape, y_test_new.shape)


# In[575]:
```

```
Type_counts = y_train_new.value_counts()

sns.barplot(x=Type_counts.index, y=Type_counts.values)
plt.xlabel('Class')
plt.ylabel('Frequency')
plt.title('Distribution of Type X (0) and Y(1) in Train Data ')
plt.show()


# In[577]:


Type_counts_test = y_test_new.value_counts()

sns.barplot(x=Type_counts_test.index, y=Type_counts_test.values)
plt.xlabel('Class')
plt.ylabel('Frequency')
plt.title('Distribution of Type X (0) and Y(1) in Test Data ')
plt.show()
data_train_new=pd.concat([X_train_new, y_train_new], axis=1)
correlation_matrix = data_train_new.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()


# In[578]:


data_train_new=pd.concat([X_train_new, y_train_new], axis=1)
correlation_matrix = data_train_new.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()
```

```
# In[580]:


data_train_new=pd.concat([X_train_new, y_train_new], axis=1)


plt.figure(figsize=(10, 6))
sns.histplot(data=data_train_new, x='Load', hue='TypeX', bins=30, kde=True)
plt.title('Distribution of Load by Type Class')
plt.xlabel('Load')
plt.ylabel('Frequency')
plt.show()


# In[583]:


# we can see in training data Loads 5-8 have just Type Y, Load 1 just has Type X , I
bins = [-np.inf, 1, 2,3,4, np.inf]  # Example bin edges, adjust these based on plot
labels = [1, 2,3, 4, 5]

# Create the binned_load feature
X_train_new['LoadCapped5'] = pd.cut(X_train_new['Load'], bins=bins, labels=labels)


# View the first few rows to confirm
print(X_train_new[['Load', 'LoadCapped5']].value_counts())



# In[585]:


# Use the same bins to transform the test set
X_test_new['LoadCapped5'] = pd.cut(X_test_new['Load'], bins=bins, labels=labels)


# In[587]:
```

```
#creating interaction between some correlated features to see if they come up more u
X_train_new['PCA_Component_1SID_S4'] = X_train_new['PCA_Component_1'] *   X_train_ne

X_test_new['PCA_Component_1SID_S4'] = X_test_new['PCA_Component_1'] *   X_test_new['


# In[1054]:


data_train_new=pd.concat([X_train_new, y_train_new], axis=1)


# In[591]:


#The newly created Interaction PCA_component1*SID_4 feature is better correlated wit
# so dropping columns Load, PCA Component 1, SID_S4

X_train_new = X_train_new.drop(columns=['Load', 'SID_S4','PCA_Component_1'])
X_test_new = X_test_new.drop(columns=['Load', 'SID_S4','PCA_Component_1'])

data_train_new=pd.concat([X_train_new, y_train_new], axis=1)
correlation_matrix = data_train_new.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()


# In[648]:


print(X_train_new['SID_S2'].unique())
print(X_train_new['SID_S3'].unique())

X_train_new['SID_S2'] = pd.factorize(X_train_new['SID_S2'])[0]
X_train_new['SID_S3'] = pd.factorize(X_train_new['SID_S3'])[0]

print(X_train_new[['SID_S2', 'SID_S3']].value_counts())
```

```
# In[650]:


X_train_plot = X_train_new.apply(pd.to_numeric, errors='coerce')

# Check for any non-numeric columns and remove them if needed
non_numeric_columns = X_train_plot.select_dtypes(exclude=['number']).columns
if len(non_numeric_columns) > 0:
    print(f"Removing non-numeric columns: {non_numeric_columns.tolist()}")
    X_train_plot = X_train_plot.drop(columns=non_numeric_columns)

# Combine X and y data to create the complete training DataFrame for plotting
data_plot_new = X_train_plot.copy()
data_plot_new[target] = y_train_new

# Define features and ensure target is in the data
features = X_train_plot.columns.tolist()

# Plotting with pairplot
pairplot = sns.pairplot(
    data_plot_new, vars=features, kind="scatter", hue=target,
    palette=['#29CFD0', '#FF0000'], plot_kws=dict(s=30, linewidth=1)
)

# Remove the legend if needed
pairplot._legend.remove()


# In[595]:


#X_train_new_pd.head()


# #### Feature Scaling

# In[593]:
```

```
# Scaling
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train_new)
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train_new.columns)
X_test_scaled = scaler.transform(X_test_new)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_train_new.columns)


# #### Feature selection

# In[597]:


# Deviance Function
def deviance(X, y, model):
    return 2 * log_loss(y, model.predict_proba(X), normalize=False)

# Subset Deviance Calculation
def subset_deviance(model, X, y, selected_ids):
    X_selected = X.iloc[:, sorted(list(selected_ids))]
    model.fit(X_selected, y)
    return deviance(X_selected, y, model)

# Feature Selection by Deviance
def select_features_by_deviance(X, y, model, max_features=None):
    n_features = X.shape[1]
    selected_ids = set()
    best_deviance = float('inf')

    for num_features in range(1, n_features + 1):
        if max_features and num_features > max_features:
            break

        for feature_ids in combinations(range(n_features), num_features):
            dev = subset_deviance(model, X, y, feature_ids)
            if dev < best_deviance:
                best_deviance = dev
                selected_ids = set(feature_ids)

    return [X.columns[i] for i in sorted(list(selected_ids))]
```

```
# Feature Selection
print("Performing feature selection based on deviance...")
selected_features = select_features_by_deviance(X_train_scaled_df, y_train_new, Logi
print(f"Selected features based on deviance: {selected_features}")


# In[609]:


# Define Model List
models = [
    {'name': 'Logistic Regression', 'model': LogisticRegression(max_iter=1000, class
    {'name': 'LinearSVC', 'model': LinearSVC(class_weight='balanced'), 'param_grid':
    {'name': 'Decision Tree', 'model': DecisionTreeClassifier(random_state=42, class
    {'name': 'SGD Classifier', 'model': SGDClassifier(random_state=42, class_weight=
    {'name': 'Bagging Classifier', 'model': BaggingClassifier(random_state=42), 'par
    {'name': 'Random Forest', 'model': RandomForestClassifier(random_state=42, class
]

# Cross-Validation
def cross_validate_model(model, X, y, param_grid, k_folds=5):
    cv = StratifiedKFold(n_splits=k_folds, shuffle=True, random_state=42)
    grid_search = GridSearchCV(model, param_grid, cv=cv, n_jobs=-1, scoring='accurac
    grid_search.fit(X, y)
    return grid_search.best_estimator_, grid_search.best_params_, grid_search.best_s

# Plot Decision Boundary (for LinearSVC, SGD, Logistic Regression)
def plot_decision_boundary(model, X, y, title):
    plt.figure()
    sns.scatterplot(x=X.iloc[:, 0], y=X.iloc[:, 1], hue=y, palette="coolwarm")
    x_min, x_max = X.iloc[:, 0].min() - 1, X.iloc[:, 0].max() + 1
    y_min, y_max = X.iloc[:, 1].min() - 1, X.iloc[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 1
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
    plt.title(f"{title} Decision Boundary")
    plt.xlabel(X.columns[0])
    plt.ylabel(X.columns[1])
```

```
    plt.show()

# Visualize Tree Structure (for Decision Tree, Bagging, Random Forest)
def plot_tree_structure(model, title):
    plt.figure(figsize=(15, 10))
    plot_tree(model, filled=True, rounded=True, feature_names=X_train.columns, max_d
    plt.title(f"{title} - Tree Structure")
    plt.show()



# Evaluate Model
def evaluate_model(model_name, best_model, X_test, y_test, X_train, y_train):
    y_pred_train = best_model.predict(X_train)
    y_pred_test = best_model.predict(X_test)
    accuracy_train = accuracy_score(y_train, y_pred_train)
    accuracy_test = accuracy_score(y_test, y_pred_test)

    print(f"\n{model_name} - Train Accuracy: {accuracy_train:.4f}, Test Accuracy: {a

    # Overfitting/Underfitting Check
    if accuracy_train > accuracy_test + 0.05:
        print(f"{model_name} is likely overfitting.")
    elif accuracy_test > accuracy_train + 0.05:
        print(f"{model_name} is likely underfitting.")
    else:
        print(f"{model_name} is well-fitted to the data.")

    # ROC Curve, Precision-Recall, Confusion Matrix, and Residuals
    y_pred_proba = best_model.decision_function(X_test) if not hasattr(best_model, '
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
    precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_pred_proba)
    roc_auc = auc(fpr, tpr)

    print("Classification Report:\n", classification_report(y_test, y_pred_test))

    # Confusion Matrix
    sns.heatmap(confusion_matrix(y_test, y_pred_test), annot=True, fmt="d", cmap="Bl
    plt.title(f"{model_name} - Confusion Matrix")
    plt.show()
```

```
    # ROC Curve
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curve')
    plt.legend(loc='lower right')
    plt.show()

    # Precision-Recall Curve
    plt.plot(recall_vals, precision_vals, label=f'{model_name}')
    plt.xlabel('Recall')
    plt.ylabel('Precision')
    plt.title('Precision-Recall Curve')
    plt.legend(loc='lower left')
    plt.show()

    # Decision Boundary (for Logistic, LinearSVC, SGD)
    if model_name in ['Logistic Regression', 'LinearSVC', 'SGD Classifier'] and X_tr
        plot_decision_boundary(best_model, X_train, y_train, model_name)

    # Tree Structure (for Decision Tree, Bagging, Random Forest)
    if model_name in ['Decision Tree', 'Bagging Classifier', 'Random Forest'] and ha
        plot_tree_structure(best_model, model_name)



# Run Cross-Validation, Evaluation, and Identify Best Model
best_model = None
best_test_accuracy = 0
best_model_name = ""
for model_dict in models:
    model_name = model_dict['name']
    best_model_cv, best_params, best_score = cross_validate_model(model_dict['model'
    print(f"\nBest params for {model_name}: {best_params}, Best CV Score: {best_scor

    evaluate_model(model_name, best_model_cv, X_test_scaled_df[selected_features], y

    # Track the best model based on test accuracy
```

```
    test_accuracy = accuracy_score(y_test_new, best_model_cv.predict(X_test_scaled_c
    if test_accuracy > best_test_accuracy:
        best_test_accuracy = test_accuracy
        best_model = best_model_cv
        best_model_name = model_name

print(f"\nThe best model is {best_model_name} with a test accuracy of {best_test_acc


# In[ ]:




# In[ ]:




# ## 3. Clustering Approach
# Now take the problem as a clustering problem { cluster data and
# then predict by the mean of viral loads of a cluster. Again, you need to design an
# clustering experiments including data preprocessing, feature engineering, model
# selection, and evaluation; conduct experiments and analyse results, making full us
# the provided d
# ter.

# In[1052]:


X_all = pd.concat([X_train_combined, X_val_combined, X_test_combined], axis=0)
y_all = pd.concat([y_train, y_val, y_test], axis=0)
data_all = pd.concat([X_all, y_all], axis=1)

# Scaling the features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_all)  # Scaling the features without the target va
```

```
# Step 2: K-Means Clustering - Finding optimal k using the Elbow method
sse = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    sse.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), sse, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('SSE')
plt.title('Elbow Method for Optimal K')
plt.show()

optimal_k = 10 # Adjust based on elbow plot

# Step 3: Clustering Models
# K-Means Clustering
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Agglomerative Clustering
agglo = AgglomerativeClustering(n_clusters=optimal_k)
agglo_labels = agglo.fit_predict(X_scaled)

# DBSCAN Clustering
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_scaled)

# Step 4: Evaluate Clustering
print("KMeans Silhouette Score:", silhouette_score(X_scaled, kmeans_labels))
print("Agglomerative Silhouette Score:", silhouette_score(X_scaled, agglo_labels))
dbscan_silhouette = silhouette_score(X_scaled, dbscan_labels) if len(set(dbscan_labe
print("DBSCAN Silhouette Score:", dbscan_silhouette)

# Step 5: Visualize Clusters using PCA
# Step 5: Visualize Clusters using PCA
pca = PCA(n_components=3)
X_pca = pca.fit_transform(X_scaled)  # Reduce the data to 3D for better visualizatio
```

```
# 3D Visualization of Clusters for KMeans, Agglomerative, DBSCAN
fig = plt.figure(figsize=(18, 5))

# KMeans Cluster Visualization in 3D
ax1 = fig.add_subplot(131, projection='3d')
ax1.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], c=kmeans_labels, cmap='coolwarm')
ax1.set_title('KMeans Clustering (PCA Reduced)')
ax1.set_xlabel('PCA Component 1')
ax1.set_ylabel('PCA Component 2')
ax1.set_zlabel('PCA Component 3')

# Agglomerative Cluster Visualization in 3D
ax2 = fig.add_subplot(132, projection='3d')
ax2.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], c=agglo_labels, cmap='coolwarm')
ax2.set_title('Agglomerative Clustering (PCA Reduced)')
ax2.set_xlabel('PCA Component 1')
ax2.set_ylabel('PCA Component 2')
ax2.set_zlabel('PCA Component 3')

# DBSCAN Cluster Visualization in 3D
ax3 = fig.add_subplot(133, projection='3d')
ax3.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], c=dbscan_labels, cmap='coolwarm')
ax3.set_title('DBSCAN Clustering (PCA Reduced)')
ax3.set_xlabel('PCA Component 1')
ax3.set_ylabel('PCA Component 2')
ax3.set_zlabel('PCA Component 3')

plt.tight_layout()
plt.show()

# Step 6: Feature Importance (Mean and Variance) within Each Cluster
# Function to calculate feature importance (mean and variance)
def feature_importance(cluster_labels, X_all):
    clusters = pd.DataFrame(cluster_labels, columns=['Cluster'])
    features = pd.concat([X_all, clusters], axis=1)

    # Mean and variance for each feature per cluster
    feature_means = features.groupby('Cluster').mean()
    feature_variances = features.groupby('Cluster').var()
```

```
    return feature_means, feature_variances

# KMeans Feature Importance
kmeans_feature_means, kmeans_feature_variances = feature_importance(kmeans_labels, X
# Agglomerative Feature Importance
agglo_feature_means, agglo_feature_variances = feature_importance(agglo_labels, X_al
# DBSCAN Feature Importance (ignoring noise points labeled as NaN)
dbscan_feature_means, dbscan_feature_variances = feature_importance(dbscan_labels, X

# Visualize Feature Means for KMeans
plt.figure(figsize=(18, 6))
sns.heatmap(kmeans_feature_means.T, cmap='viridis', annot=True)
plt.title('KMeans Feature Importance (Mean per Cluster)')
plt.show()

# Visualize Feature Means for Agglomerative Clustering
plt.figure(figsize=(18, 6))
sns.heatmap(agglo_feature_means.T, cmap='viridis', annot=True)
plt.title('Agglomerative Feature Importance (Mean per Cluster)')
plt.show()

# Visualize Feature Means for DBSCAN
plt.figure(figsize=(18, 6))
sns.heatmap(dbscan_feature_means.T, cmap='viridis', annot=True)
plt.title('DBSCAN Feature Importance (Mean per Cluster)')
plt.show()

# Visualize Feature Variances for KMeans
plt.figure(figsize=(18, 6))
sns.heatmap(kmeans_feature_variances.T, cmap='viridis', annot=True)
plt.title('KMeans Feature Variance per Cluster')
plt.show()



# Step 8: Dendrogram for Agglomerative Clustering
Z = linkage(X_scaled, 'ward')

plt.figure(figsize=(10, 7))
```

```
dendrogram(Z)
plt.title("Dendrogram for Agglomerative Clustering")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

# Step 9: DBSCAN Evaluation over a range of eps values
eps_values = np.linspace(0.1, 1.0, 10)
dbscan_silhouette_scores = []

for eps in eps_values:
    dbscan = DBSCAN(eps=eps, min_samples=5)
    dbscan_labels = dbscan.fit_predict(X_scaled)

    # Only evaluate if there are more than 1 cluster (excluding noise)
    if len(set(dbscan_labels)) > 1:
        score = silhouette_score(X_scaled, dbscan_labels)
    else:
        score = -1  # Not valid, since DBSCAN found only 1 cluster or noise

    dbscan_silhouette_scores.append(score)

# Plot silhouette scores for DBSCAN
plt.figure(figsize=(8, 5))
plt.plot(eps_values, dbscan_silhouette_scores, marker='o')
plt.xlabel('Eps Values')
plt.ylabel('Silhouette Score')
plt.title('DBSCAN Silhouette Score for Different Eps Values')
plt.show()

# Step 10: Best Parameters Log for KMeans, Agglomerative, and DBSCAN

# KMeans best k (this was already determined in Step 2)
best_k = optimal_k  # Use the optimal_k from earlier step

# Agglomerative Clustering best k (this was also determined in Step 2)
agglo_best_k = optimal_k  # Use the optimal_k from earlier step

# DBSCAN with optimal epsilon (based on the silhouette score from earlier)
eps_best = eps_values[np.argmax(dbscan_silhouette_scores)]
```

```
# Output the best parameters for each clustering method
print(f"Best K for KMeans: {best_k} with Silhouette Score: {silhouette_score(X_scale
print(f"Best K for Agglomerative: {agglo_best_k} with Silhouette Score: {silhouette_
print(f"Best Eps for DBSCAN: {eps_best} with Silhouette Score: {max(dbscan_silhouett

# Print the number of clusters identified by DBSCAN (including noise labeled as -1)
dbscan_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)  # Exc
print(f"Number of clusters identified by DBSCAN (excluding noise): {dbscan_clusters}


# Step 7: Calculate Mean Viral Load for Each Cluster and Evaluate MSE
def predict_viral_load_by_cluster(cluster_labels, y_actual):
    # Map each cluster to the mean viral load in that cluster
    cluster_to_load_mean = {label: y_actual[cluster_labels == label].mean() for labe

    # Generate predicted viral load based on assigned cluster's mean
    predicted_loads = np.array([cluster_to_load_mean[label] for label in cluster_lab

    return predicted_loads, cluster_to_load_mean

# Calculate predictions and MSE for each clustering model
kmeans_predicted, kmeans_load_means = predict_viral_load_by_cluster(kmeans_labels, y
agglo_predicted, agglo_load_means = predict_viral_load_by_cluster(agglo_labels, y_al
dbscan_predicted, dbscan_load_means = predict_viral_load_by_cluster(dbscan_labels, y

# Check that predicted arrays match y_all in length before computing MSE
print("y_all length:", len(y_all))
print("kmeans_predicted length:", len(kmeans_predicted))
print("agglo_predicted length:", len(agglo_predicted))
print("dbscan_predicted length:", len(dbscan_predicted))

# Calculate and print MSE for each model
kmeans_mse = mean_squared_error(y_all, kmeans_predicted)
agglo_mse = mean_squared_error(y_all, agglo_predicted)
dbscan_mse = mean_squared_error(y_all, dbscan_predicted)

print(f"KMeans MSE: {kmeans_mse}")
print(f"Agglomerative MSE: {agglo_mse}")
print(f"DBSCAN MSE: {dbscan_mse}")
```

```
# Create DataFrame to display mean load per cluster for each model
# Step 7: Calculate Mean Viral Load for Each Cluster and Evaluate MSE
def predict_viral_load_by_cluster(cluster_labels, y_actual):
    # Map each cluster to the mean viral load in that cluster
    cluster_to_load_mean = {label: y_actual[cluster_labels == label].mean() for labe

    # Generate predicted viral load based on assigned cluster's mean
    predicted_loads = np.array([cluster_to_load_mean[label] for label in cluster_lab

    return predicted_loads, cluster_to_load_mean

# Calculate predictions and MSE for each clustering model
kmeans_predicted, kmeans_cluster_means = predict_viral_load_by_cluster(kmeans_labels
agglo_predicted, agglo_cluster_means = predict_viral_load_by_cluster(agglo_labels, y
dbscan_predicted, dbscan_cluster_means = predict_viral_load_by_cluster(dbscan_labels

# Print mean viral load per cluster for each model
print("KMeans Cluster Means:")
kmeans_means_df = pd.DataFrame.from_dict(kmeans_cluster_means, orient='index', colum
print(kmeans_means_df)

print("\nAgglomerative Cluster Means:")
agglo_means_df = pd.DataFrame.from_dict(agglo_cluster_means, orient='index', columns
print(agglo_means_df)

print("\nDBSCAN Cluster Means:")
dbscan_means_df = pd.DataFrame.from_dict(dbscan_cluster_means, orient='index', colum
print(dbscan_means_df)

# Step 8: Boxplot for Mean Load of Each Cluster by Model
plt.figure(figsize=(15, 5))

# KMeans Boxplot
plt.subplot(1, 3, 1)
sns.boxplot(x=kmeans_labels, y=y_all, palette="viridis")
plt.title("KMeans Mean Viral Load per Cluster")
plt.xlabel("KMeans Cluster")
plt.ylabel("Viral Load")
```

```
# Agglomerative Clustering Boxplot
plt.subplot(1, 3, 2)
sns.boxplot(x=agglo_labels, y=y_all, palette="viridis")
plt.title("Agglomerative Mean Viral Load per Cluster")
plt.xlabel("Agglomerative Cluster")
plt.ylabel("Viral Load")

# DBSCAN Boxplot
plt.subplot(1, 3, 3)
sns.boxplot(x=dbscan_labels, y=y_all, palette="viridis")
plt.title("DBSCAN Mean Viral Load per Cluster")
plt.xlabel("DBSCAN Cluster")
plt.ylabel("Viral Load")

plt.tight_layout()
plt.show()
```